

Multi-Block Diffusion Language Models

Yijie Jin¹, Jiajun Xu², Yuxuan Liu¹, Chenkai Xu¹, Yi Tu³, Jiajun Li³, Dandan Tu³, Xiaohui Yan³, Kai Yu¹, Pengfei Liu¹, Zhijie Deng^{1,†}

¹Shanghai Jiao Tong University, ²Xi'an Jiao Tong University, ³Huawei

Block Diffusion Language Models (BD-LMs) improve diffusion-based text generation with KV caching and flexible-length generation. A natural next step is to extend them from Single-Block Diffusion (SingleBD) to Multi-Block Diffusion (MultiBD), where a *running-set* of consecutive blocks is decoded concurrently for inter-block parallelism. However, existing BD-LMs are mostly trained under teacher forcing, where the model observes only one noisy block conditioned on a clean prefix. While the recent diffusion forcing strategy introduces visibility among multiple noisy blocks, its training states still differ from MultiBD inference, where decoding operates on a bounded *running-set* with heterogeneous slot-wise noise patterns. To bridge this gap, we propose *Multi-Block Diffusion Language Models* (MBD-LMs), obtained by post-training BD-LMs with *Multi-block Teacher Forcing* (MultiTF). MultiTF integrates teacher forcing and diffusion forcing by training on bounded *noise-groups* conditioned on clean prefixes, with randomized *noise-schedulers* that better match MultiBD inference states. To make MultiBD practically executable, we further introduce an optimized decoding algorithm based on the *Block Buffer* mechanism that preserves prefix-cache reuse, keeps input shapes static, and translates increased decoding parallelism into wall-clock acceleration. Empirically, MBD-LLaDA2-Mini increases average Tokens Per Forward pass (TPF) from 3.47 to **6.19** and improves average accuracy from 79.95% to **81.03%**; when combined with DMax, MBD-LLaDA2-Mini-DMax reaches an average TPF of **9.34** with only a 1.02% accuracy drop on math and code benchmarks.

Training Code: <https://github.com/SJTU-DENG-Lab/mbd-lms>

Inference Code: <https://github.com/SJTU-DENG-Lab/Diffulex>

Correspondence: Zhijie Deng: zhijied@sjtu.edu.cn

Contributions: [†] Corresponding author.

Date: June 21, 2026

1 Introduction

Diffusion Language Models (DLMs) have emerged as a promising alternative to autoregressive language models by enabling native parallel decoding (Sahoo et al., 2024; Nie et al., 2025). However, fully bidirectional DLMs struggle to serve efficiently because they lack support for KV caching and dynamic-length generation.

Recent Block Diffusion Language Models (BD-LMs) have become a representative DLM paradigm for efficient generation, addressing the above limitations through block-causal generation (Arriola et al., 2025; Bie et al., 2025; Cheng et al., 2025). Most BD-LMs trained under Teacher Forcing (TF) naturally support *Single-Block Diffusion* (SingleBD): at each forward pass, the model decodes one noisy block while preceding blocks are already clean and cached, enabling KV caching and intra-block parallelism. However, blocks themselves are still processed sequentially. As shown in Figure 1, SingleBD must finish decoding a block and storing its KV cache before later blocks can proceed, creating storing bubbles and locking inter-block parallelism.

The Discrete Diffusion Forcing (D2F) (Wang et al., 2025) strategy introduces the visibility of multiple noisy blocks to BD-LMs. Conditioned on a clean prefix, it corrupts suffix blocks with monotonic increasing noise ratios during training. Consequently, D2F obtains *Multi-Block Diffusion* (MultiBD) capability, as shown in Figure 1, enabling decode-store overlap and inter-block parallelism. However, a train-inference mismatch problem remains. Specifically, it is not possible to process the entire noisy suffix as one *running-set* in a single forward pass, from both the perspectives of efficiency and empirical efficacy (Lu et al., 2026). For the naive MultiBD introduced by D2F, the expected *running-set* size is often around two, and adjacent slots exhibit large noise-ratio gaps. This suggests that reliable MultiBD requires training states that match both the bounded *running-set* size and the heterogeneous slot-wise noise patterns observed during inference.

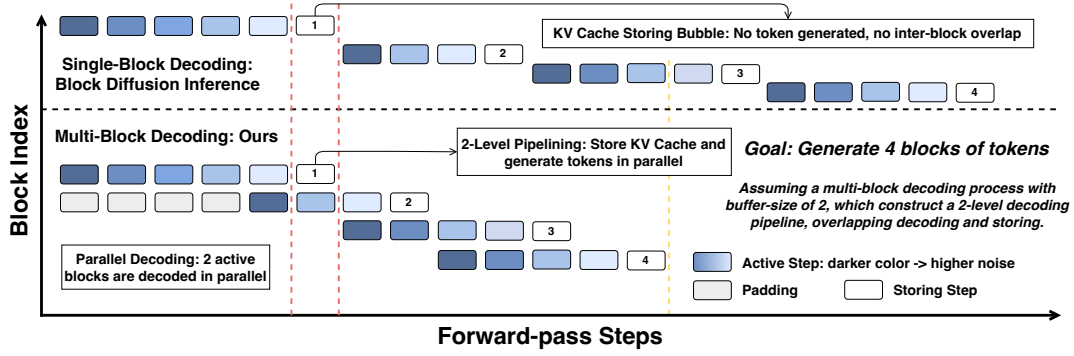


Figure 1 SingleBD decodes blocks sequentially and creates KV cache storing bubbles. In contrast, MultiBD overlaps future-block refinement with KV cache storing of completed blocks, and enables inter-block parallelism.

To this end, we formulate *Multi-Block Diffusion Language Models* (MBD-LMs), a unified view of existing BD-LMs. This view covers both TF-trained BD-LMs and D2F-trained BD-LMs as extreme cases, while identifying practical MultiBD as the bounded intermediate regime for reliable and efficient inference.

We introduce *Multi-block Teacher Forcing* (MultiTF), a post-training method that turns BD-LMs into MBD-LMs. MultiTF extends TF by concatenating the clean prefix with a bounded group of consecutive noisy blocks, where noisy blocks can attend to each other under a *Group-Aware Dual-Stream Mask*. It applies a more aggressive and randomized *noise-scheduler* within each *noise-group* to simulate the heterogeneous slot-wise noise patterns observed during inference. During training, blocks are partitioned into groups with varying sizes to cover possible *running-set* sizes and group-relative positions.

We further propose an optimized inference pipeline for MultiBD. MultiBD relies on a dynamic *running-set* for decoding, which is unfriendly to CUDA Graph capture and replay. To address this, we introduce the *Block Buffer* mechanism, which maintains a fixed number of block slots. Future blocks enter the *Block Buffer* by activating existing idle slots rather than extending the physical input, while completed front blocks leave after being committed to the KV cache. This design keeps the input shape static, preserves KV caching and prefix caching, and translates the increased TPF into practical wall-clock speedup.

Experiments on math and code benchmarks show that MBD-LMs improve decoding parallelism while preserving generation quality. Compared with LLaDA2-Mini (Bie et al., 2025), **MBD-LLaDA2-Mini** increases the average TPF from 3.47 to **6.19** (+78.4%) and improves the average accuracy from 79.95% to **81.03%**. When combined with DMax (Chen et al., 2026), **MBD-LLaDA2-Mini-DMax** further reaches an average TPF of **9.34** (+47.1% over LLaDA2-Mini-DMax under SingleBD) with only a 1.02 percentage-point accuracy drop. Using our inference engine, MBD-LLaDA2-Mini-DMax achieves 951.41 TPS on average, compared with 781.50 TPS for LLaDA2-Mini-DMax.

Main Contributions

- ✓ **Unified MBD-LM formulation.** We formulate *Multi-Block Diffusion Language Models* (MBD-LMs) as a unified DLM framework parameterized by a *running-set* of consecutive blocks. This view covers both TF-trained BD-LMs and D2F-trained BD-LMs, while identifying practical MultiBD as the bounded intermediate regime for reliable and efficient inference.
- ✓ **MultiTF post-training for MBD-LMs.** We propose Multi-block Teacher Forcing (MultiTF), a post-training method that turns BD-LMs into MBD-LMs. MultiTF improves train-inference alignment by training BD-LMs on states that resemble practical MultiBD inference.
- ✓ **Optimized MultiBD inference engine.** We design and implement an optimized MultiBD inference pipeline based on the *Block Buffer* mechanism. The pipeline overlaps decoding and KV cache storing, preserves prefix caching, and keeps input shapes static for CUDA Graph capture and replay, translating increased TPF into practical TPS gains.

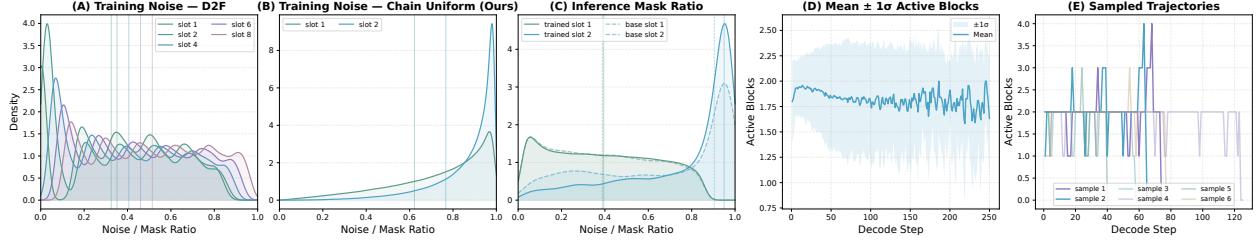


Figure 2 Train-inference statistics for MultiBD. **(A)** Slot-wise mask-ratio distributions induced by the D2F-style monotonic scheduler. **(B)** Slot-wise mask-ratio distributions induced by our chain-uniform scheduler. **(C)** Inference-time mask-ratio distributions before and after MultiTF post-training. **(D)** Mean and one-standard-deviation range of the active-block count during MultiBD inference. **(E)** Sampled active-block trajectories during decoding. Panels (A–C) compare scheduler-induced training noise patterns with inference-time mask-ratio patterns for train-inference alignment analysis. Panels (D–E) report the active part of the MultiBD *running-set* under a buffer size of four; the active-block count can therefore occasionally exceed two.

2 Preliminaries

2.1 Diffusion Language Models

Diffusion Language Models (DLMs) (Sahoo et al., 2024; Nie et al., 2025; Ye et al., 2025) formulate text generation as iterative denoising. Let $\mathcal{V}$ denote the vocabulary, $[\mathbf{M}]$ denote a special mask token, and L denote the sequence length. Given a clean sequence $\mathbf{x}_0 = (x_0^1, \dots, x_0^L) \in \mathcal{V}^L$, the forward process gradually masks tokens independently. For $t \in [0, 1]$, the noisy sequence $\mathbf{x}_t \in (\mathcal{V} \cup \{[\mathbf{M}]\})^L$ masks each token with probability t :

$$q_t(x_t^i | x_0^i) = \begin{cases} 1 - t, & x_t^i = x_0^i, \\ t, & x_t^i = [\mathbf{M}], \\ 0, & \text{otherwise.} \end{cases} \quad (2.1)$$

Let $\mathcal{M}(\mathbf{x}_t) = \{i : x_t^i = [\mathbf{M}]\}$ denote the masked positions. A DLM parameterized by θ predicts clean tokens at masked positions:

$$p_\theta(\mathbf{x}_0 | \mathbf{x}_t) = \prod_{i=1}^L p_\theta(x_0^i | \mathbf{x}_t). \quad (2.2)$$

The standard training objective is a weighted masked-token cross-entropy (Nie et al., 2025):

$$\mathcal{L}_{\text{DLM}}(\theta) = -\mathbb{E}_{t, \mathbf{x}_0, \mathbf{x}_t} \left[\frac{1}{t} \sum_{i=1}^L \mathbf{1}[x_t^i = [\mathbf{M}]] \cdot \log p_\theta(x_0^i | \mathbf{x}_t) \right], \quad (2.3)$$

where $t \sim \mathcal{U}(0, 1)$, $\mathbf{x}_t \sim q_t(\cdot | \mathbf{x}_0)$, and $\mathbf{1}[\cdot]$ denotes the indicator function, ensuring that the loss is computed only on masked tokens. The inference starts from an all- $[\mathbf{M}]$ sequence and iteratively fills high-confidence masked positions.

2.2 Block Diffusion Language Models

Block Diffusion Language Models (BD-LMs) (Arriola et al., 2025; Bie et al., 2025) partition the sequence into blocks, i.e.,

$$\mathbf{x}_0 = [\mathbf{b}_1, \dots, \mathbf{b}_K], \quad \mathbf{b}_k \in \mathcal{V}^B, \quad (2.4)$$

where B is the block size and $K = L/B$ is the number of blocks. BD-LMs model the sequence autoregressively at the block level:

$$p_\theta(\mathbf{x}_0) = \prod_{k=1}^K p_\theta(\mathbf{b}_k | \mathbf{x}_0^{(<k)}), \quad \mathbf{x}_0^{(<k)} = [\mathbf{b}_1, \dots, \mathbf{b}_{k-1}]. \quad (2.5)$$

Each conditional term is implemented by a DLM decoding process within the current block. The block-causal attention pattern is used to allow each block to attend to itself and preceding blocks. This enables KV caching during *Single-Block Diffusion* (SingleBD) inference.

Teacher forcing. Block Diffusion (Arriola et al., 2025) trains BD-LMs under Teacher Forcing (TF). For block $\mathbf{b}_k$, only the current block is corrupted by the same masking process,

$$\mathbf{b}_{k,t} \sim q_t(\cdot | \mathbf{b}_k), \quad (2.6)$$

and the model predicts masked tokens conditioned on clean prefix blocks:

$$\mathcal{L}_{\text{TF}}(\theta) = -\mathbb{E}_{k,t,\mathbf{x}_0,\mathbf{b}_{k,t}} \left[\frac{1}{t} \sum_{i=1}^B \mathbf{1}[b_{k,t}^i = [\mathbf{M}]] \cdot \log p_{\theta}(b_k^i | \mathbf{x}_0^{(<k)}, \mathbf{b}_{k,t}) \right]. \quad (2.7)$$

Namely, the model only learns to decode one noisy block conditioned on clean prefix blocks, which is conceptually incompatible with the aforementioned MultiBD inference.

Discrete diffusion forcing. Another training paradigm for BD-LMs is Discrete Diffusion Forcing (D2F) (Wang et al., 2025). D2F introduces visibility among noisy blocks by sampling block-level noise ratios $\mathbf{t} = (t_1, \dots, t_K)$ for a block-partitioned suffix.

Let

$$\mathbf{x}_0^{\text{pre}} = (x_0^1, \dots, x_0^P) \in \mathcal{V}^P$$

denote a clean token-level prefix of length P , and let

$$\mathbf{x}_0^{\text{suf}} = [\mathbf{b}_1, \dots, \mathbf{b}_K], \quad \mathbf{b}_k \in \mathcal{V}^B,$$

denote the suffix partitioned into blocks. D2F constructs noisy suffix blocks

$$\mathbf{x}_{\mathbf{t}}^{\text{suf}} = [\mathbf{b}_{1,t_1}, \dots, \mathbf{b}_{K,t_K}], \quad \mathbf{b}_{k,t_k} \sim q_{t_k}(\cdot | \mathbf{b}_k), \quad (2.8)$$

where $0 \leq t_1 < \dots < t_K \leq 1$. Thus, earlier suffix blocks are less masked, while later suffix blocks are more uncertain. Conditioned on the clean prefix, D2F trains the student to predict each suffix block from a noisy-prefix view:

$$p_{\theta}(\mathbf{x}_0^{\text{suf}} | \mathbf{x}_0^{\text{pre}}, \mathbf{x}_{\mathbf{t}}^{\text{suf}}) = \prod_{k=1}^K p_{\theta}(\mathbf{b}_k | \mathbf{x}_0^{\text{pre}}, \mathbf{b}_{1,t_1}, \dots, \mathbf{b}_{k,t_k}). \quad (2.9)$$

In practice, D2F is trained with an asymmetric distillation paradigm (Wang et al., 2025).

Despite the goal to perform *Multi-Block Diffusion* (MultiBD), D2F still differs from MultiBD inference in its training states, as detailed in Section 3.1. Beyond the aforementioned mismatch, native D2F also raises a prefix-caching concern. Its clean prefix $\mathbf{x}_0^{\text{pre}}$ can have arbitrary length P and is processed with full attention rather than block-causal attention. Therefore, its native formulation is not directly compatible with the prefix caching of BD-LMs. We analyze this issue in Appendix C.5, where we compare native D2F with a fully block-causal D2F variant and show that enforcing cache compatibility causes a larger quality degradation, further motivating MultiTF.

3 Methodology

3.1 Multi-Block Diffusion Language Models

Multi-Block Diffusion (MultiBD) generalizes the standard BD-LM factorization in Equation 2.5 by allowing a *running-set* of consecutive blocks to be decoded concurrently. At decoding step s , MultiBD maintains a *running-set*

$$\mathcal{R}_s = \{a_s, \dots, c_s\},$$

where a_s and c_s denote the first and last block indices that have not yet entered the prefix KV cache. The *running-set* contains the real blocks currently involved in MultiBD decoding, including active noisy blocks and completed preceding blocks waiting to be cached. Blocks before the *running-set* have already been committed and form the clean cached prefix:

$$\mathbf{x}_0^{(<a_s)} = [\mathbf{b}_1, \dots, \mathbf{b}_{a_s-1}].$$

For each block $k \in \mathcal{R}_s$, let $t_{k,s} \in [0, 1]$ denote its current mask ratio at decoding step s . If block k is still active, $\mathbf{b}_{k,t_{k,s}}$ is its current noisy state. If block k is completed but not yet cached, we set $t_{k,s} = 0$, so that $\mathbf{b}_{k,t_{k,s}} = \mathbf{b}_{k,0} = \mathbf{b}_k$. We refer to each relative block position inside $\mathcal{R}_s$ as a logical slot; for example, the block at index a_s is the first slot and the block at index $a_s + 1$ is the second slot.

We define *Multi-Block Diffusion Language Models* (MBD-LMs) as:

$$p_\theta(\mathbf{b}_{\mathcal{R}_s} \mid \mathbf{x}_0^{(<a_s)}, \mathbf{b}_{\mathcal{R}_s, \mathbf{t}_s}) = \prod_{k=a_s}^{c_s} p_\theta(\mathbf{b}_k \mid \mathbf{x}_0^{(<a_s)}, \mathbf{b}_{a_s, t_{a_s, s}}, \dots, \mathbf{b}_{k, t_{k, s}}), \quad (3.1)$$

where

$$\mathbf{b}_{\mathcal{R}_s} = [\mathbf{b}_{a_s}, \dots, \mathbf{b}_{c_s}], \quad \mathbf{b}_{\mathcal{R}_s, \mathbf{t}_s} = [\mathbf{b}_{a_s, t_{a_s, s}}, \dots, \mathbf{b}_{c_s, t_{c_s, s}}].$$

This formulation asks the model to recover the current *running-set* from the clean cached prefix and the visible block states inside $\mathcal{R}_s$. The *running-set* size is defined as $|\mathcal{R}_s|$.

The *running-set* view gives a unified way to describe existing BD-LM regimes. As illustrated in Figure 3, TF-trained BD-LMs correspond to the SingleBD extreme, where the model only observes one noisy block conditioned on a clean cached prefix. D2F-trained BD-LMs introduce visibility among multiple noisy suffix blocks, but their training states still differ from practical MultiBD inference in *running-set* size and slot-wise noise patterns. Under the MBD-LM formulation, these regimes can be viewed as limiting cases, while practical MultiBD is the bounded intermediate regime that decodes a small *running-set* concurrently.

Conceptually, MultiBD reduces to SingleBD when $|\mathcal{R}_s| = 1$: the model decodes only one block conditioned on the clean cached prefix. At the other extreme, if the *running-set* is expanded to cover all suffix blocks and a monotonic D2F-style *noise-scheduler* is used, the resulting training state resembles the fully block-causal D2F variant discussed in Appendix C.5. This connection is only at the level of training-state construction: D2F remains a training paradigm, while MultiBD is the inference regime targeted by MBD-LMs. In practice, useful MultiBD operates between these two extremes: $|\mathcal{R}_s|$ should be larger than 1 to expose inter-block parallelism, but remain bounded to keep each forward pass efficient and executable. This bounded *running-set* view is consistent with the empirical MultiBD traces analyzed in Section 4.4, and is reflected in both the training-side and inference-side designs proposed below.

3.2 Multi-block Teacher Forcing

Multi-block Teacher Forcing (MultiTF) post-trains BD-LMs into MBD-LMs by constructing inference-like training states, with particular emphasis on matching the bounded *running-set* structure and the slot-wise noise patterns of MultiBD inference. MultiTF can be viewed as an extension of TF from one noisy block to a bounded group of consecutive noisy blocks. We call such a group a *noise-group*. Following the bounded

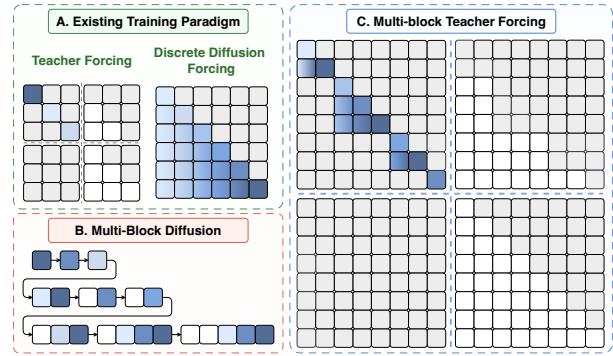


Figure 3 Train-inference alignment across paradigms. (A) TF and D2F provide existing BD-LM training states, but neither matches practical MultiBD. (B) MultiBD maintains a bounded *running-set* for concurrent block refinement. (C) MultiTF builds inference-like *noise-groups* with heterogeneous slot-wise noise patterns.

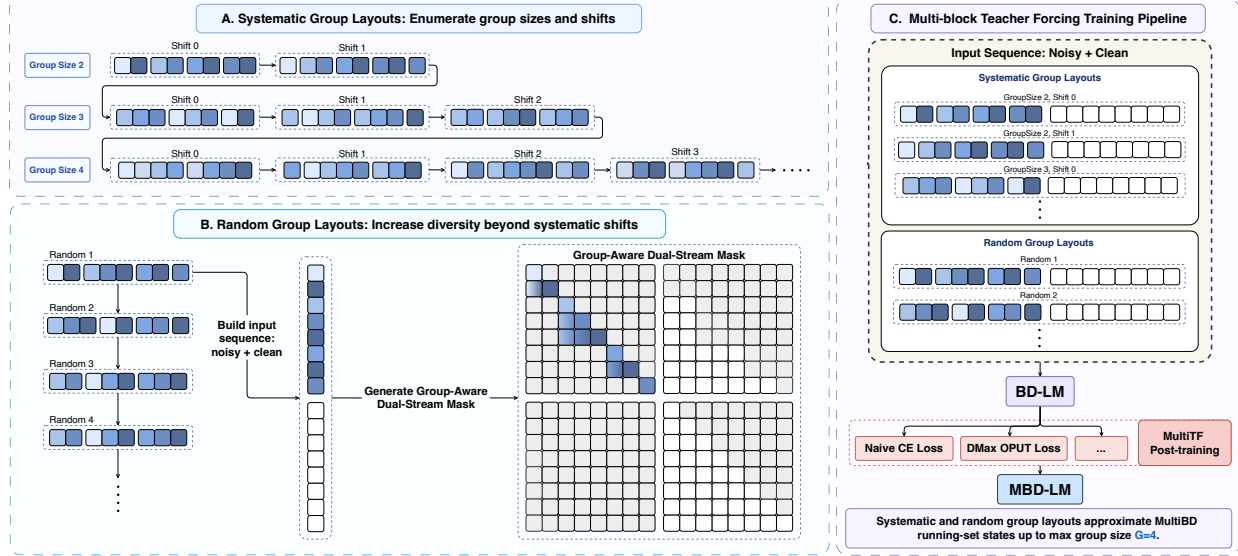


Figure 4 Overview of MultiTF. **(A)** Systematic *group-layouts* enumerate group sizes and shifts so that blocks appear at different group-relative positions. **(B)** Random *group-layouts* increase layout diversity; each layout is converted into a noisy-clean input sequence with the *Group-Aware Dual-Stream Mask*. **(C)** The resulting input sequences are used to post-train BD-LMs into MBD-LMs with masked CE and optional model-specific objectives.

running-set view in Section 3.1, MultiTF uses $G_{\max}$ as the training-side upper bound on *noise-group* size. Throughout the paper, $G_{\max}$ denotes the maximum *noise-group* size, Λ denotes the set of sampled *group-layouts*, $\lambda \in \Lambda$ denotes one layout, and H_m denotes one *noise-group*. Each *noise-group* H_m is constructed as a bounded training analogue of a possible MultiBD *running-set*. Notably, later *noise-groups* are conditioned on *clean* earlier *noise-groups* during training.

Here $\mathcal{J}$ is only the finite-layout estimator accumulated inside Algorithm 1; the population-level training objective is $\mathcal{L}_{\text{MultiTF}}$ in Equation 3.4.

Group-layout construction. Given a clean block sequence $[\mathbf{b}_1, \dots, \mathbf{b}_K]$, MultiTF constructs a set of *group-layouts* Λ , where each *group-layout* $\lambda = (H_1, \dots, H_{|\lambda|})$ partitions the sequence into consecutive *noise-groups*. Each *noise-group* $H_m = \{a_m, \dots, c_m\}$ has the same consecutive-block form as a possible MultiBD *running-set* $\mathcal{R}_s = \{a_s, \dots, c_s\}$. We use both systematic and random *group-layouts* to cover different bounded *running-set* sizes and group-relative positions, as shown in Figure 4.

- **Systematic layouts.** We specify a maximum *noise-group* size $G_{\max}$. For each *noise-group* size $g \in \{2, \dots, G_{\max}\}$ and each shift $h \in \{0, \dots, g-1\}$, we define a shifted layout $\lambda_{g,h}$ by placing group boundaries every g blocks with offset h :

$$H_{g,h,q} = \{1 + h + qg, \dots, h + (q+1)g\} \cap \{1, \dots, K\},$$

where q indexes groups within the shifted layout, and boundary groups are clipped to the valid block range. The systematic layout set is

$$\Lambda_{\text{sys}} = \{\lambda_{g,h} : g \in \{2, \dots, G_{\max}\}, h \in \{0, \dots, g-1\}\}.$$

This construction ensures that, ignoring boundary effects, every consecutive *running-set* $\{a, \dots, a+g-1\}$ of length g appears as one *noise-group* in exactly one shifted layout, with shift $h = (a-1) \bmod g$. Equivalently, for each fixed g , every block appears once at every group-relative position across the g shifts.

- **Random layouts.** Systematic layouts provide structured coverage but are regular by construction. To increase layout diversity, we further sample random layouts by drawing *noise-group* sizes $g_m \in$

Algorithm 1 Multi-block Teacher Forcing

Require: Clean sequence $\mathbf{x}_0$; block size B ; maximum *noise-group* size $G_{\max}$; noise bounds $t_{\text{low}}, t_{\text{high}}$; margin ratio ρ ; number of random layouts N_{rand} ; mask token $[\mathbf{M}]$.

// **Construct noise-group layouts**

- 1: Partition $\mathbf{x}_0$ into K blocks $[\mathbf{b}_1, \dots, \mathbf{b}_K]$.
- 2: Generate systematic layouts by enumerating *noise-group* sizes $g \in \{2, \dots, G_{\max}\}$ and all g group shifts.
- 3: Generate N_{rand} random layouts by sampling *noise-group* sizes from $\{2, \dots, G_{\max}\}$ until all blocks are covered.
- 4: Let Λ be the union of systematic and random layouts.

// **Apply MultiTF corruption and training**

- 5: Set $t_{\text{eff}} \leftarrow t_{\text{high}} - \rho(t_{\text{high}} - t_{\text{low}})$, where ρ is the noise-transition margin ratio.
- 6: Initialize accumulated loss $\mathcal{J} \leftarrow 0$.
- 7: **for** each layout $\lambda \in \Lambda$ **do**
- 8: Initialize noisy sequence $\mathbf{x}_t^\lambda \leftarrow \mathbf{x}_0$.
- 9: **for** each *noise-group* $H_m = (j_1, \dots, j_{n_m}) \in \lambda$ **do**
- 10: // **Chain-uniform block-level noise-scheduler**
- 11: Sample group floor $\ell \sim \mathcal{U}(t_{\text{low}}, t_{\text{eff}})$.
- 12: **for** $i \leftarrow 1$ **to** n_m **do**
- 13: Sample $t_{j_i} \sim \mathcal{U}(\ell, t_{\text{eff}})$ and set $\ell \leftarrow t_{j_i}$.
- 14: Mask $\lfloor B \cdot t_{j_i} \rfloor$ random positions in $\mathbf{b}_{j_i}$ as $[\mathbf{M}]$.
- 15: **end for**
- 16: **end for**
- 17: // **Build input sequence and attention mask**
- 18: Construct $\mathbf{X}_\lambda = [\mathbf{x}_t^\lambda; \mathbf{x}_0]$.
- 19: Construct the *Group-Aware Dual-Stream Mask* $\mathbf{A}_\lambda$.
- 20: Run the model on $(\mathbf{X}_\lambda, \mathbf{A}_\lambda)$.
- 21: // **Compute masked CE**
- 22: Let $\mathcal{M}_\lambda = \{i : \mathbf{x}_t^\lambda[i] = [\mathbf{M}]\}$.
- 23: Compute layout-level masked CE estimate $\mathcal{J}_\lambda$ over $\mathcal{M}_\lambda$.
- 24: $\mathcal{J} \leftarrow \mathcal{J} + \mathcal{J}_\lambda$.
- 25: **end for**
- 26: **return** $\mathcal{J}/|\Lambda|$.

$\{2, \dots, G_{\max}\}$ and forming consecutive groups

$$H_m = \{a_m, \dots, \min(a_m + g_m - 1, K)\}, \quad a_{m+1} = \min(a_m + g_m, K + 1),$$

until the full sequence is covered. These random layouts add non-regular *noise-group-size* combinations and boundary patterns without replacing the coverage guarantee of systematic layouts.

The final layout set is

$$\Lambda = \Lambda_{\text{sys}} \cup \Lambda_{\text{rand}}.$$

We provide a theoretical coverage view in Appendix A, showing how systematic shifts cover bounded *running-sets* while random layouts add distributional diversity.

Chain-uniform noise-scheduling. After sampling a *group-layout*, MultiTF assigns mask ratios within each *noise-group*. Unlike D2F’s monotonic block-level schedule over a long noisy sequence, MultiTF uses a randomized chain-uniform *noise-scheduler* inside each bounded *noise-group*. Specifically, for each *noise-group*, we sample a group-level floor and then sample each block’s mask ratio with the previous block’s ratio as the lower bound, as shown in Algorithm 1. This produces monotonic but randomized group-internal noise levels, encouraging larger slot-wise noise gaps that better match MultiBD inference.

Group-Aware Dual-Stream Mask. For each layout λ , the sampled block-level mask ratios corrupt the clean sequence into a noisy sequence $\mathbf{x}_t^\lambda$. Following the TF-style construction (Arriola et al., 2025), MultiTF builds the input sequence by concatenating the noisy sequence with the clean sequence:

$$\mathbf{X}_\lambda = [\mathbf{x}_t^\lambda; \mathbf{x}_0]. \quad (3.2)$$

The noisy part represents the MultiBD-like decoding state, while the clean part provides clean-prefix context.

We construct a *Group-Aware Dual-Stream Mask* over $\mathbf{X}_\lambda$:

$$\mathbf{A}_\lambda = \begin{bmatrix} \mathbf{M}_{\text{GD}} & \mathbf{M}_{\text{GOC}} \\ 0 & \mathbf{M}_{\text{BC}} \end{bmatrix}, \quad (3.3)$$

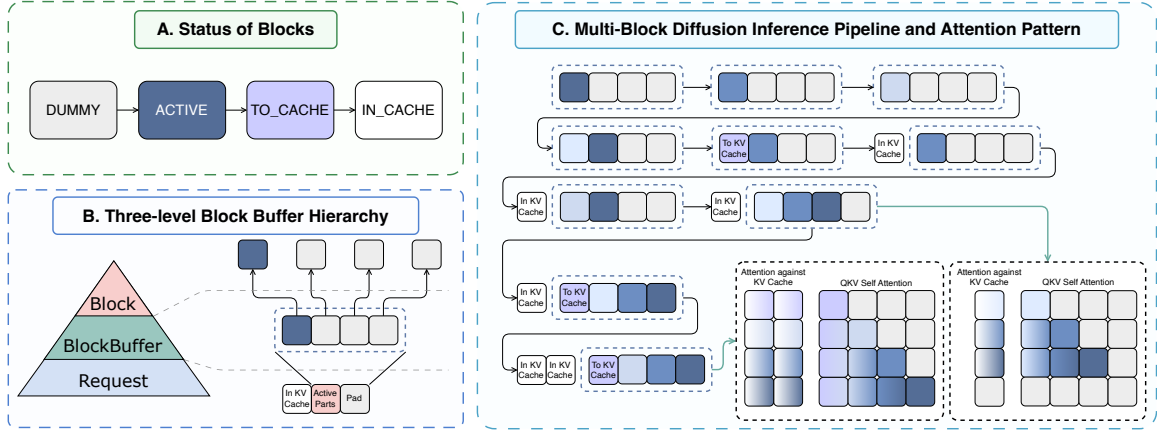


Figure 5 Inference and system support in MultiBD. (1) Blocks follow a four-state transition: DUMMY $\rightarrow$ ACTIVE $\rightarrow$ TO-CACHE $\rightarrow$ IN-CACHE. (2) MultiBD organizes decoding with a block-buffer-request hierarchy, where each request maintains *Block Buffers* and each buffer contains multiple block slots for parallel refinement. (3) During MultiBD inference, noisy blocks are refined jointly under block-causal self-attention, while committed prefix blocks are served from the KV cache; completed blocks enter the cache and the *Block Buffer* slides forward.

where $\mathbf{M}_{\text{GD}}$ enables group-internal noisy-block visibility, $\mathbf{M}_{\text{GOC}}$ lets each *noise-group* condition on its clean prefix, and $\mathbf{M}_{\text{BC}}$ preserves standard block-causal visibility on the clean part. The zero lower-left block prevents clean tokens from attending to noisy tokens; detailed mask definitions are provided in Appendix B.

Training objective. MultiTF optimizes masked-token cross-entropy on the noisy part of the input sequence:

$$\mathcal{L}_{\text{MultiTF}} = -\mathbb{E}_{\lambda, \mathbf{t}, \mathbf{x}_0} \left[\frac{1}{|\mathcal{M}_\lambda|} \sum_{i \in \mathcal{M}_\lambda} \log p_\theta(x_0^i | \mathbf{X}_\lambda, \mathbf{A}_\lambda) \right], \quad (3.4)$$

where

$$\mathcal{M}_\lambda = \{i : \mathbf{x}_t^\lambda[i] = [\mathbf{M}]\} \quad (3.5)$$

denotes masked positions on the noisy part. All systematic and random layouts are batched as independent input sequences, as illustrated in Figure 4. For models with additional objectives, such as DMax, we apply the corresponding model-specific loss on top of the same MultiTF inputs.

The concrete MultiTF objective and model-specific training variants are detailed in Appendix B.4.

3.3 Optimized Multi-Block Diffusion

After MultiTF post-training, an MBD-LM performs MultiBD inference over the *running-set* $\mathcal{R}_s$ in Equation 3.1. The inference objective is to expose inter-block parallelism without losing the serving advantages of BD-LMs. Concretely, practical MultiBD should satisfy the following inference requirements:

Inference Requirements for Practical MultiBD

- **Inter-block parallelism:** multiple noisy blocks are decoded in parallel.
- **Decode-store overlap:** decoding of later active blocks overlaps with KV cache storing of completed preceding blocks.
- **Prefix-cache preservation:** committed prefix blocks should produce stable KV cache that remains reusable by the standard BD-LM prefix cache.
- **Static-shape execution:** the physical input shape remains fixed for CUDA Graph capture and replay and efficient execution.

Naive MultiBD and dynamic execution. A naive block-causal MultiBD implementation naturally supports inter-block parallelism and decode-store overlap. As illustrated in Figure 1 and detailed in Algorithm 4, it directly materializes the *running-set* $\mathcal{R}_s$ as the input to each forward pass: future noisy blocks are appended to $\mathcal{R}_s$ when the latest active block makes sufficient progress, and completed preceding blocks are removed after being cached. Thus, later blocks can already be decoded while earlier completed blocks are being stored, avoiding the storing bubbles of SingleBD. This dynamic procedure only needs three logical block states,

$$\text{ACTIVE} \rightarrow \text{TO-CACHE} \rightarrow \text{IN-CACHE},$$

because every block in the *running-set* corresponds to a real block being decoded or committed. However, since each forward pass is built directly from $\mathcal{R}_s$, the number of processed tokens changes over time and across requests, making CUDA Graph capture and replay difficult.

Static-shape execution with Block Buffer. To satisfy all four requirements simultaneously, we decouple the logical *running-set* from the physical input by using a *Block Buffer* mechanism, as detailed in Algorithm 5. As shown in Figure 5(B), our inference engine organizes MultiBD decoding with a three-level hierarchy: a request manages one or more *Block Buffers*, each *Block Buffer* contains a fixed number of block slots, and each slot stores one block state. The request level handles generation progress and cache ownership, the *Block Buffer* level provides a static physical input for CUDA Graph replay, and the block level tracks whether each slot is DUMMY, ACTIVE, TO-CACHE, or IN-CACHE.

Let $\mathcal{W}_s$ denote the physical *Block Buffer* at decoding step s . It contains a fixed number of block slots:

$$|\mathcal{W}_s| = N_{\text{buf}},$$

where N_{buf} is the buffer size. The real resident blocks inside $\mathcal{W}_s$ form the *running-set* $\mathcal{R}_s$, while the remaining slots are dummy slots. Thus, the buffer can be written as

$$\mathcal{W}_s = \mathcal{R}_s \parallel \mathcal{D}_s, \quad |\mathcal{W}_s| = |\mathcal{R}_s| + |\mathcal{D}_s| = N_{\text{buf}}, \quad |\mathcal{R}_s| \leq N_{\text{buf}},$$

where $\mathcal{D}_s$ denotes the trailing dummy segment. Thus, N_{buf} is the inference-side realization of the bounded *running-set* assumption introduced in Section 3.1. In practice, N_{buf} is chosen within the *running-set* sizes covered by MultiTF through G_{max} .

A future block enters decoding by activating an existing dummy slot rather than extending the physical input sequence. When the front block of $\mathcal{R}_s$ is completed, it is marked as TO-CACHE; once committed to the KV cache, it leaves $\mathcal{R}_s$ and becomes part of the cached prefix. The *Block Buffer* then slides forward by appending a new dummy slot at the tail. Thus, MultiBD can advance its *running-set* while keeping the physical buffer shape fixed, thereby enabling static-shape execution for CUDA Graph capture and replay.

As shown in Figure 5(A), each physical slot follows the state transition

$$\text{DUMMY} \rightarrow \text{ACTIVE} \rightarrow \text{TO-CACHE} \rightarrow \text{IN-CACHE}.$$

The key difference from the naive three-state dynamic procedure is the additional DUMMY state, which reserves inactive capacity inside the *Block Buffer*. This allows future blocks to enter by activating existing slots instead of extending the physical input, while completed front blocks are committed into the KV cache.

Prefix-cache preservation. The *Block Buffer* mechanism also preserves the cache semantics of block-causal BD-LMs. Committed front blocks become immutable clean prefix blocks and are represented only through cached KV states, while active blocks remain inside the *Block Buffer* for iterative refinement. This separation is important because native D2F uses prefix-full attention and is not directly compatible with the standard BD-LM prefix-cache interface, as discussed in Section 2.2. Appendix C.5 further shows that simply converting D2F into a fully block-causal variant improves cache compatibility but causes a larger quality degradation. In contrast, MultiTF trains MBD-LMs with block-causal clean-prefix conditioning, and the *Block Buffer* inference pipeline preserves this prefix-cache interface during MultiBD decoding.

This design preserves inter-block parallelism, overlaps decoding with KV cache storing, maintains prefix-cache reuse, and supports static-shape execution for CUDA Graph replay. As a result, the increased TPF of MBD-LMs can be converted into practical wall-clock speedup. Additional implementation details, including the

Table 1 Evaluation results across math and code benchmarks. SingleBD (Native) denotes the native single-block diffusion inference of each BD-LM; MultiBD (training-free) applies multi-block decoding without retraining; MBD-* denotes the corresponding MultiTF-post-trained MBD-LM. AUP (Accuracy Under Parallelism) combines accuracy and TPF, reported in the **Average** column as an aggregate across four benchmarks. MBD-LMs consistently improve TPF over SingleBD. In most settings, MultiTF recovers or improves the quality lost by training-free MultiBD, leading to a better accuracy-parallelism trade-off.

Model	GSM8K		MATH500		MBPP+		HumanEval+		Average		
	Acc ↑	TPF ↑	Acc ↑	TPF ↑	Acc ↑	TPF ↑	Acc ↑	TPF ↑	Acc ↑	TPF ↑	AUP ↑
LLaDA2-Mini-DMax (<i>bufsz=2, blkksz=32</i>)											
SingleBD (Native)	91.89	5.70	76.80	6.13	72.22	6.14	77.44	7.44	79.59	6.35	459.54
MultiBD (training-free)	89.84	8.76	73.80	9.08	72.22	8.44	<u>76.83</u>	10.96	78.17	<u>9.31</u>	<u>651.98</u>
MBD-LLaDA2-Mini-DMax	<u>91.74</u>	8.95	<u>75.00</u>	9.31	<u>70.11</u>	<u>8.34</u>	77.44	<u>10.78</u>	<u>78.57</u>	9.34	661.28
LLaDA2-Mini (<i>bufsz=2, blkksz=32</i>)											
SingleBD (Native)	91.89	2.27	<u>74.20</u>	2.83	75.66	3.25	<u>78.05</u>	5.53	<u>79.95</u>	3.47	247.41
MultiBD (training-free)	92.65	<u>2.76</u>	73.60	<u>3.53</u>	<u>72.49</u>	<u>3.97</u>	75.61	<u>7.37</u>	78.59	<u>4.41</u>	<u>301.81</u>
MBD-LLaDA2-Mini	<u>91.96</u>	5.55	79.20	6.02	<u>72.49</u>	5.35	80.49	7.85	81.03	6.19	449.18
SDAR-8B-Chat-b32 (<i>bufsz=4, blkksz=32</i>)											
SingleBD (Native)	90.07	2.52	<u>65.60</u>	3.81	<u>52.65</u>	1.83	67.68	2.00	<u>69.00</u>	2.54	141.64
MultiBD (training-free)	89.01	<u>2.78</u>	60.60	<u>5.06</u>	<u>52.12</u>	<u>1.97</u>	<u>65.85</u>	<u>2.24</u>	66.89	<u>3.01</u>	<u>156.35</u>
MBD-SDAR-8B-Chat-b32	<u>89.16</u>	3.08	68.00	5.08	58.99	4.87	62.80	4.82	69.74	4.46	210.42
SDAR-8B-Chat-b4 (<i>bufsz=4, blkksz=4</i>)											
SingleBD (Native)	<u>91.05</u>	1.33	72.80	1.46	<u>64.80</u>	1.13	<u>73.70</u>	1.07	75.59	1.25	85.46
MultiBD (training-free)	90.45	2.39	70.60	2.68	65.80	<u>1.55</u>	74.39	<u>1.47</u>	<u>75.31</u>	<u>2.00</u>	<u>129.59</u>
MBD-SDAR-8B-Chat-b4	91.81	<u>2.28</u>	<u>72.40</u>	<u>2.52</u>	64.29	2.62	72.56	2.24	75.27	2.42	148.65

(a) Training-free MultiBD transfers to additional model variants. SingleBD (Native) denotes each model’s native single-block diffusion inference.

	GSM8K		MATH500		Average		
	Acc ↑	TPF ↑	Acc ↑	TPF ↑	Acc ↑	TPF ↑	AUP ↑
LLaDA2-Mini-CAP (<i>bufsz=2, blkksz=32</i>)							
SingleBD (Native)	91.74	3.08	77.80	3.71	84.77	3.40	247.30
MultiBD (training-free)	91.21	4.00	77.20	4.94	84.21	4.47	319.17
LLaDA2.1-Mini (<i>bufsz=2, blkksz=32</i>)							
SingleBD (Native)	93.03	4.12	81.40	4.87	87.22	4.50	390.64
MultiBD (training-free)	92.27	5.80	81.00	7.20	86.63	6.50	558.52

(b) Ablation of MultiTF training components averaged over HumanEval+ and GSM8K with LLaDA2-Mini-DMax.

Configuration	Acc ↑	TPF ↑	AUP ↑
SingleBD (Native)	84.67	6.57	536.89
noise-group layouts construction			
+ systematic layouts	83.22	9.71	<u>774.03</u>
+ random layouts	82.72	9.42	747.46
systematic + random layouts (ours)	<u>84.59</u>	9.87	805.34
block-level noise-scheduler			
D2F-style monotonic scheduler	79.34	8.76	657.74
random scheduler	83.14	9.70	771.74
sorted-uniform scheduler	81.28	<u>9.73</u>	748.73
chain-uniform scheduler (ours)	<u>84.59</u>	9.87	805.34

Table 2 Transfer and ablation results. (a) Training-free MultiBD transfers to additional model variants on math benchmarks. (b) MultiTF component ablations averaged over HumanEval+ and GSM8K. All reported metrics are higher-is-better.

naive dynamic MultiBD, the optimized MultiBD, block-state transitions, threshold rules, and prefix-cache analysis, are provided in Appendix C. The realized speedup is validated by the TPS results in Table 3.

4 Experiments

4.1 Experimental Setup

Models and training. We evaluate MultiTF on representative BD-LMs from the LLaDA2.x (Bie et al., 2025, 2026) and SDAR (Cheng et al., 2025) families, including variants enhanced with DMax (Chen et al., 2026). For each base model, MultiTF post-training constructs multiple *group-layouts* per sample, including systematic shifted layouts and random layouts, to approximate the MultiBD *running-set* states described in Section 3.1. The resulting models are denoted as MBD-* models, e.g., MBD-LLaDA2-Mini and MBD-SDAR-8B-Chat. We also evaluate training-free MultiBD, which directly applies MultiBD inference to the original BD-LMs without post-training.

Benchmarks and metrics. We evaluate mathematical reasoning on GSM8K (Cobbe et al., 2021) and

MATH500 (Hendrycks et al., 2021), and code generation on MBPP+ and HumanEval+ (Liu et al., 2023). We report Accuracy, Tokens Per Forward pass (TPF), and Accuracy Under Parallelism (AUP). Accuracy is exact match for math and pass@1 for code. TPF measures decoding parallelism, while AUP summarizes the accuracy–parallelism trade-off following d3LLM (Qian et al., 2026). Given a set of decoding configurations $\mathcal{C}$, we sort them by TPF and compute AUP as the trapezoidal area under the accuracy–TPF curve:

$$\text{AUP} = \sum_{i=1}^{|\mathcal{C}|-1} \frac{A_{c_i} + A_{c_{i+1}}}{2} (P_{c_{i+1}} - P_{c_i}), \quad (4.1)$$

where A_{c_i} and P_{c_i} denote the accuracy and TPF of configuration c_i , respectively. For multi-benchmark evaluation, we report the average AUP across benchmarks.

Experimental details. Detailed training hyperparameters, inference hyperparameters, hardware settings, and training costs are provided in Appendix D.

4.2 Main Results

We first evaluate whether MBD-LMs can improve decoding parallelism without sacrificing generation quality. The analysis focuses on four questions: (i) whether MultiTF-post-trained MBD-LMs improve the TPF–accuracy trade-off over native SingleBD; (ii) whether MultiTF is complementary to T2T-enhanced decoding methods such as DMax; (iii) whether train–inference alignment is necessary beyond training-free MultiBD; and (iv) whether the gains generalize across different BD-LM backbones.

Baselines and configurations. Table 1 reports results across four benchmarks. For each base BD-LM, we compare three configurations: (1) **SingleBD (Native)**, the model’s native single-block diffusion inference; (2) **MultiBD (training-free)**, MultiBD inference applied without post-training; and (3) **MBD-***, the corresponding MultiTF-post-trained model using MultiBD inference.

Main analysis. MBD-LMs improve decoding parallelism while preserving generation quality. Compared with LLaDA2-Mini under SingleBD (Native), MBD-LLaDA2-Mini increases average TPF from 3.47 to **6.19** (+78.4%) and improves average accuracy from 79.95% to **81.03%**. Notably, even without DMax, MBD-LLaDA2-Mini reaches a TPF comparable to LLaDA2-Mini-DMax under SingleBD (6.19 vs. 6.35), while achieving higher average accuracy (81.03% vs. 79.59%). This shows that MultiTF can turn a standard BD-LM into an MBD-LM with DMax-level decoding parallelism.

Compatibility with T2T-enhanced decoding. MultiTF is complementary to DMax, a Token-to-Token (T2T) enhanced acceleration method. When combined with DMax, MBD-LLaDA2-Mini-DMax further increases average TPF from 6.35 to **9.34** (+47.1%) over LLaDA2-Mini-DMax under SingleBD, with only a 1.02 percentage-point average accuracy drop. This indicates that MBD-LMs can stack with existing T2T-enhanced recipes.

Effect of train–inference alignment. The comparison between training-free MultiBD and MultiTF-post-trained MBD-LMs highlights the importance of train–inference alignment. Directly applying MultiBD already increases TPF, confirming that multi-block decoding relaxes the single-block bottleneck. However, it can degrade accuracy because the original BD-LMs are not trained on practical MultiBD states. MultiTF reduces this mismatch: on LLaDA2-Mini, accuracy improves from 78.59% under training-free MultiBD to **81.03%** after MultiTF post-training, while average TPF further increases from 4.41 to **6.19**. On LLaDA2-Mini-DMax, MultiTF improves average accuracy from 78.17% to **78.57%** while preserving high TPF.

Generalization across BD-LM backbones. MBD-LMs also generalize beyond the LLaDA2 family. On SDAR-8B-Chat-b32, MBD-SDAR-8B-Chat-b32 increases average TPF from 2.54 to **4.46** (+75.6%) and improves average accuracy from 69.00% to **69.74%**. With block size 4, MBD-SDAR-8B-Chat-b4 reaches the best average AUP among the three SDAR configurations. These results suggest that the MBD-LM formulation and MultiTF post-training are not tied to a specific BD-LM backbone.

Transfer of training-free MultiBD. In addition, Table 2a shows that training-free MultiBD transfers to additional model variants such as LLaDA2-Mini-CAP and LLaDA2.1-Mini, improving TPF without post-training. This suggests that the inference-side MultiBD mechanism itself has broad applicability, while MultiTF is needed to recover and further improve generation quality under practical MultiBD states.

4.3 Ablation Study

Table 2b ablates the key MultiTF training components with LLaDA2-Mini-DMax, averaged over HumanEval+ and GSM8K. Compared with SingleBD (Native), the full MBD configuration increases TPF from 6.57 to **9.87** and AUP from 536.89 to **805.34**, while nearly preserving the average accuracy, with only a 0.08-point change from 84.67% to 84.59%. This shows that MultiTF substantially improves the TPF–accuracy trade-off by aligning BD-LMs with practical MultiBD inference states.

Effect of noise-group group-layouts. We first ablate the *group-layout* construction for *noise-groups*. Using only systematic layouts or only random layouts already improves TPF over SingleBD, increasing TPF from 6.57 to 9.71 and 9.42, respectively. However, both single-source variants reduce accuracy, with systematic layouts achieving 83.22% and random layouts achieving 82.72%. Combining systematic and random layouts gives the best trade-off, reaching the highest TPF of **9.87** and the highest AUP of **805.34**, while recovering the accuracy to 84.59%, close to the SingleBD level of 84.67%. This suggests that the two layout sources are complementary: systematic *group-layouts* provide structured coverage of bounded *running-set* sizes and group-relative positions, while random *group-layouts* add distributional diversity beyond the systematic construction.

Effect of block-level noise-schedulers. We then ablate the block-level *noise-scheduler* within each *noise-group*. Replacing the chain-uniform *noise-scheduler* with a D2F-style monotonic *noise-scheduler* increases TPF over SingleBD from 6.57 to 8.76, but causes a large accuracy drop from 84.67% to 79.34%. This indicates that exposing the model to multiple noisy blocks is insufficient when the slot-wise noise pattern is not aligned with practical MultiBD inference. Random and sorted-uniform *noise-schedulers* further improve TPF to 9.70 and 9.73, respectively, but still underperform chain-uniform in AUP. In particular, sorted-uniform achieves a high TPF but suffers a larger accuracy drop, suggesting that sorted mask ratios alone do not capture the heterogeneous noise gaps induced by MultiBD decoding. The full chain-uniform *noise-scheduler* achieves the best accuracy, TPF, and AUP among the scheduler variants, reaching 84.59%, 9.87, and 805.34, respectively. This confirms the importance of training with heterogeneous slot-wise noise gaps. The sorted-uniform *noise-scheduler* baseline samples mask ratios uniformly and sorts them before assigning them to slots; details are provided in Appendix B. We further analyze the train–inference alignment gap in Section 4.4.

4.4 Train–Inference Alignment Analysis

Figure 2 analyzes the training-state mismatch that motivates MultiTF. The figure focuses on two aspects of practical MultiBD inference: slot-wise mask-ratio patterns and the size of the active part of the *running-set*.

D2F-style noise schedules mismatch MultiBD inference. As shown in Figure 2(A), the D2F-style monotonic scheduler induces highly overlapping slot-wise mask-ratio distributions. This weak slot-wise separation differs from practical MultiBD inference, where adjacent active slots often exhibit large noise-ratio gaps. This explains the ablation result in Table 2b: the D2F-style monotonic *noise-scheduler* improves TPF by enabling multi-block decoding, but causes a large accuracy drop because its training states do not match practical MultiBD inference states.

Chain-uniform scheduling improves slot-wise alignment. By contrast, the chain-uniform scheduler used by MultiTF creates more heterogeneous slot-wise noise patterns. As shown in Figure 2(B), different slots in a *noise-group* receive more separated mask-ratio distributions. These scheduler-induced training distributions better match the inference-time mask-ratio distributions in Figure 2(C), especially the large gap between the first and second active slots. After MultiTF post-training, the inference-time mask-ratio distribution becomes further aligned with the designed training states.

MultiBD inference uses a bounded active set. Figure 2(D-E) further shows that MultiBD inference usually maintains a small active part of the *running-set*, with an expectation around two and occasional expansion to three or four active blocks. This supports the bounded *running-set* view in Section 3.1. Reliable MultiBD therefore requires training states that match both the bounded *running-set* structure and the heterogeneous slot-wise noise patterns of inference, rather than merely exposing the model to future noisy blocks.

4.5 Efficiency Analysis

We further analyze how the increased TPF of MBD-LMs translates into realized wall-clock throughput. At decoding step s , the optimized MultiBD engine executes a fixed physical *Block Buffer* $\mathcal{W}_s$ defined in Section 3.3. Let P_s denote the cached prefix length at this step and let

$$Q_s = |\mathcal{W}_s|B = N_{\text{buf}}B$$

denote the number of processed tokens in one forward pass. For SingleBD, this reduces to $N_{\text{buf}} = 1$ and $Q_s = B$. For MultiBD, $N_{\text{buf}} > 1$, and the forward pass processes all physical buffer slots, including active blocks, completed resident blocks, and dummy slots used to preserve static input shapes. Thus, Q_s measures the computational workload of a forward pass, whereas TPF measures the number of useful tokens committed by that forward pass.

This distinction defines a token-efficiency factor:

$$\eta_{\text{tok}}(s) = \frac{\text{TPF}_s}{Q_s}.$$

Equivalently,

$$\text{TPS} = \frac{\text{TPF}}{T_{\text{step}}} = \frac{\eta_{\text{tok}}Q_s}{T_{\text{step}}}.$$

Therefore, increasing the block-buffer size can improve throughput only when the useful-token gain outweighs the additional per-step cost. MultiBD increases Q_s and enables more tokens to be committed per forward pass, but its token efficiency can be reduced by inactive dummy slots and resident blocks that are processed for static-shape execution but do not immediately contribute to committed tokens.

Each decoding forward can be viewed as an extend-attention step with Q_s query tokens and a cached prefix of length P_s . For a transformer with N_{layer} layers, hidden size d , FFN hidden size d_{ff} , and vocabulary $\mathcal{V}$, the per-step FLOPs can be approximated as

$$\mathcal{F}_{\text{step}}(Q_s, P_s) = \Theta \left(N_{\text{layer}} [Q_s(d^2 + dd_{\text{ff}}) + d(Q_sP_s + Q_s^2)] + Q_sd|\mathcal{V}| \right).$$

The first term comes from QKV/O projections and FFN layers, the second term comes from attention between the buffer and the cached prefix as well as attention inside the buffer, and the last term comes from the LM head when logits are computed. Thus, increasing N_{buf} from 1 to a larger value improves inter-block decoding parallelism, but also increases the amount of computation performed by each forward pass.

The memory cost follows the same extend-attention structure. Let s_{dtype} be the number of bytes per activation element. The per-step weight traffic scales as

$$\mathcal{M}_W = \Theta \left(N_{\text{layer}}s_{\text{dtype}}(d^2 + dd_{\text{ff}}) \right),$$

while the KV-cache traffic of extend attention can be approximated as

$$\mathcal{M}_{\text{KV}}(Q_s, P_s) = \Theta \left(N_{\text{layer}}s_{\text{dtype}} [\rho(Q_s)(P_s + Q_s)d + Q_sd] \right),$$

where $\rho(Q_s)$ captures repeated KV reads caused by query tiling. The first term corresponds to reading KV cache for the prefix and current buffer, while the second term corresponds to KV cache storing.

This gives a roofline-style view of the step latency:

$$T_{\text{step}}(Q_s, P_s) \approx \max \left(\frac{\mathcal{F}_{\text{step}}(Q_s, P_s)}{\Pi_{\text{eff}}}, \frac{\mathcal{M}_W + \mathcal{M}_{\text{KV}}(Q_s, P_s)}{\mathcal{B}_{\text{HBM}}} \right) + T_{\text{comm}}(Q_s) + T_{\text{launch}},$$

Table 3 Throughput and single-step latency comparison. Results are measured for single-sample decoding on two H100 GPUs with tensor parallelism degree 2 (TP=2). Step latency denotes the average wall-clock latency of one decoding forward pass. TPF and TPS gains are computed relative to LLaDA2-Mini, while latency cost reports the relative increase in per-step latency.

Model	Forward-step statistics					Realized throughput					
	Avg. TPF ↑	TPF Gain ↑	Step Lat. (ms) ↓	Lat. Cost ↓		GSM8K TPS ↑	MATH500 TPS ↑	MBPP+ TPS ↑	HumanEval+ TPS ↑	Avg. TPS ↑	TPS Gain ↑
LLaDA2-Mini	3.47	–	7.07	1.00×		344.05	403.45	496.19	824.94	517.16	–
MBD-LLaDA2-Mini	6.19	+78.39%	8.78	1.24×		687.87	707.89	646.73	941.18	745.92	+44.24%
LLaDA2-Mini-DMax	6.35	+83.00%	9.02	1.28×		700.82	730.60	754.97	931.55	779.49	+50.73%
MBD-LLaDA2-Mini-DMax	9.34	+169.16%	11.20	1.58×		834.52	851.07	896.65	1124.43	926.67	+79.19%

where Π_{eff} is the effective compute throughput, $\mathcal{B}_{\text{HBM}}$ is the effective HBM bandwidth, T_{comm} includes fixed-configuration tensor-parallel communication, and T_{launch} denotes launch and runtime overhead. This expression shows that the realized throughput depends on both the useful-token numerator and the roofline-limited per-step cost denominator.

The attention arithmetic intensity further explains why MultiBD can still be efficient despite processing more tokens per step. Ignoring lower-order terms, the attention arithmetic intensity is approximately

$$\text{AI}_{\text{attn}} \approx \frac{dQ_s P_s}{s_{\text{dtype}} \rho(Q_s) P_s d} = \Theta \left(\frac{Q_s}{s_{\text{dtype}} \rho(Q_s)} \right)$$

when $P_s \gg Q_s$. Therefore, increasing Q_s through a larger *Block Buffer* makes the extend-attention step more compute intensive. Prefix KV reads, weight reads, and kernel-launch overheads are amortized over more query tokens. However, the gain is useful only to the extent that these processed tokens lead to committed tokens, as captured by η_{tok} .

The measurements in Table 3 match this analysis. For LLaDA2-Mini, MBD increases the average TPF from 3.47 to 6.19, a $1.78\times$ improvement, while the step latency increases from 7.07 ms to 8.78 ms, a $1.24\times$ cost increase. The expected throughput scaling is therefore approximately $1.78/1.24 = 1.44\times$, closely matching the measured Avg. TPS improvement from 517.16 to 745.92, i.e., $1.44\times$. Similarly, for LLaDA2-Mini-DMax, MBD increases the average TPF from 6.35 to 9.34, a $1.47\times$ improvement, while the step latency increases from 9.02 ms to 11.20 ms, a $1.24\times$ cost increase. This predicts a throughput scaling of $1.47/1.24 = 1.18\times$, which closely matches the measured Avg. TPS improvement from 779.49 to 926.67, i.e., $1.19\times$. Thus, the observed gap between TPF gain and TPS gain is primarily explained by the increased per-forward cost of processing the larger static *Block Buffer*.

Overall, MultiBD improves wall-clock throughput by increasing the number of useful tokens committed per forward pass and by making each extend-attention step more compute intensive. At the same time, static-shape execution introduces extra processed tokens through resident blocks and dummy slots, reducing token efficiency relative to the ideal case. The final TPS gain is therefore determined by the balance among TPF improvement, token efficiency, and roofline-limited step latency.

5 Related Work

5.1 Diffusion Language Models

Diffusion Language Models (DLMs) generate text through iterative denoising and enable parallel token refinement as an alternative to autoregressive generation. Representative models include LLaDA (Nie et al., 2025), Dream (Ye et al., 2025), and LLaDA2.x (Bie et al., 2025, 2026), which improve scaling, initialization, and editable refinement. However, fully bidirectional DLMs are difficult to serve efficiently because they do not naturally support KV caching or flexible-length generation.

Block Diffusion Language Models (BD-LMs) (Arriola et al., 2025; Bie et al., 2025; Cheng et al., 2025) address these limitations by introducing block-causal generation. Their native *Single-Block Diffusion* (SingleBD) inference decodes one noisy block conditioned on a clean cached prefix, enabling KV caching and intra-block parallel decoding. Nevertheless, SingleBD still processes blocks sequentially, leaving inter-block parallelism

underused. Our work studies *Multi-Block Diffusion* (MultiBD) as a broader inference regime for BD-LMs, where a bounded *running-set* of consecutive blocks can be refined concurrently.

5.2 Efficient DLM Inference and Training

Efficient DLMs have been studied through distillation, scheduling, caching, and parallel decoding. D2F (Wang et al., 2025) introduces noisy-block visibility during training and demonstrates the potential of MultiBD-style pipelined decoding. DMax (Chen et al., 2026), d3LLM (Qian et al., 2026), LightningRL (Hu et al., 2026), and dParallel (Chen et al., 2025) improve the accuracy-parallelism trade-off through training objectives or decoding schedules. Fast-dLLM (Wu et al., 2025) and LoPA (Xu et al., 2025) accelerate inference through caching and lookahead parallelism.

Our work is complementary to these efforts but focuses on a different level of parallelism. Instead of only increasing token-level parallelism or applying MultiBD as an inference-time heuristic, we treat MultiBD as a target inference regime for BD-LMs. We identify the bounded *running-set* structure and heterogeneous slot-wise noise patterns as key train-inference alignment factors, and propose MultiTF to post-train BD-LMs into MBD-LMs with inference-like multi-block states. We further provide *Block Buffer* inference support so that MultiBD preserves prefix-cache reuse and static-shape execution.

6 Conclusion

We proposed *Multi-Block Diffusion Language Models* (MBD-LMs), a unified formulation of BD-LMs for reliable MultiBD inference. Starting from the sequential bottleneck of SingleBD, we showed that MultiBD can expose inter-block parallelism but requires training states aligned with its bounded *running-set* structure and heterogeneous slot-wise noise patterns. To bridge this gap, we introduced *Multi-block Teacher Forcing* (MultiTF), which post-trains BD-LMs with bounded *noise-groups*, the *Group-Aware Dual-Stream Mask*, and randomized block-level *noise-schedulers*. We further developed an optimized MultiBD inference engine with the *Block Buffer* mechanism, enabling static-shape execution while preserving KV caching and prefix-cache reuse. Experiments on math and code benchmarks show that MBD-LMs improve decoding parallelism and realized throughput while maintaining generation quality, demonstrating that reliable MultiBD requires both training-time state alignment and inference-time system support.

References

- Marianne Arriola, Aaron Gokaslan, Justin T. Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. In *International Conference on Learning Representations (ICLR)*, 2025. URL <https://arxiv.org/abs/2503.09573>. Oral Presentation.
- Tiwei Bie, Zenan Huang, Chongxuan Li, et al. Llada2.0: Scaling up diffusion language models to 100b. *arXiv preprint arXiv:2512.15745*, 2025. URL <https://arxiv.org/abs/2512.15745>.
- Tiwei Bie et al. Llada2.1: Speeding up text diffusion via token editing. *arXiv preprint arXiv:2602.08676*, 2026. URL <https://arxiv.org/abs/2602.08676>.
- Nicolas Boizard, Hippolyte Gisserot-Boukhlef, Kevin El-Haddad, Céline Hudelot, and Pierre Colombo. When does reasoning matter? a controlled study of reasoning’s contribution to model performance. *arXiv preprint arXiv:2509.22193*, 2025. URL <https://arxiv.org/abs/2509.22193>.
- Zigeng Chen, Gongfan Fang, Xinyin Ma, Ruonan Yu, and Xinchao Wang. dparallel: Learnable parallel decoding for dllms. *arXiv preprint arXiv:2509.26488*, 2025. URL <https://arxiv.org/abs/2509.26488>.
- Zigeng Chen, Gongfan Fang, Xinyin Ma, Ruonan Yu, and Xinchao Wang. Dmax: Aggressive parallel decoding for dllms. *arXiv preprint arXiv:2604.08302*, 2026. URL <https://arxiv.org/abs/2604.08302>.
- Shuang Cheng, Yihan Bian, Dawei Liu, Linfeng Zhang, Qian Yao, Zhongbo Tian, Wenhai Wang, Qipeng Guo, Kai Chen, Biqing Qi, and Bowen Zhou. Sdar: A synergistic diffusion-autoregression paradigm for scalable sequence generation. *arXiv preprint arXiv:2510.06303*, 2025. URL <https://arxiv.org/abs/2510.06303>.

- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.
- Yanzhe Hu, Yijie Jin, Pengfei Liu, Kai Yu, and Zhijie Deng. Lightningrl: Breaking the accuracy-parallelism trade-off of block-wise dlms via reinforcement learning. *arXiv preprint arXiv:2603.13319*, 2026. URL <https://arxiv.org/abs/2603.13319>.
- jtatman. Python code dataset 500k. Hugging Face dataset, 2025. URL <https://huggingface.co/datasets/jtatman/python-code-dataset-500k>.
- Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems*, 36:21558–21572, 2023.
- Guanxi Lu, Hao Mark Chen, Yuto Karashima, Zhican Wang, Daichi Fujiki, and Hongxiang Fan. Adablock-dllm: Semantic-aware diffusion llm inference via adaptive block size. *arXiv preprint arXiv:2509.26432*, 2026. URL <https://arxiv.org/abs/2509.26432>.
- Qianli Ma, Yaowei Zheng, Zhelun Shi, Zhongkai Zhao, Bin Jia, Ziyue Huang, Zhiqi Lin, Youjie Li, Jiacheng Yang, Yanghua Peng, Zhi Zhang, and Xin Liu. Veomni: Scaling any modality model training with model-centric distributed recipe zoo. *arXiv preprint arXiv:2508.02317*, 2025. URL <https://arxiv.org/abs/2508.02317>.
- Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. *arXiv preprint arXiv:2502.09992*, 2025. URL <https://arxiv.org/abs/2502.09992>.
- Yu-Yang Qian, Junda Su, Lanxiang Hu, Peiyuan Zhang, Zhijie Deng, Peng Zhao, and Hao Zhang. d3llm: Ultra-fast diffusion llm using pseudo-trajectory distillation. *arXiv preprint arXiv:2601.07568*, 2026. URL <https://arxiv.org/abs/2601.07568>.
- Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T. Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. *arXiv preprint arXiv:2406.07524*, 2024. URL <https://arxiv.org/abs/2406.07524>.
- Xu Wang, Chenkai Xu, Yijie Jin, Jiachun Jin, Hao Zhang, and Zhijie Deng. Diffusion llms can do faster-than-ar inference via discrete diffusion forcing. *arXiv preprint arXiv:2508.09192*, 2025. URL <https://arxiv.org/abs/2508.09192>.
- Chengyue Wu et al. Fast-dllm: Training-free acceleration of diffusion llm by enabling kv cache and parallel decoding. *arXiv preprint arXiv:2505.22618*, 2025. URL <https://arxiv.org/abs/2505.22618>.
- Chenkai Xu, Yijie Jin, Jiajun Li, Yi Tu, Guoping Long, Dandan Tu, Mingcong Song, Hongjie Si, Tianqi Hou, Junchi Yan, and Zhijie Deng. Lopa: Scaling dllm inference via lookahead parallel decoding. *arXiv preprint arXiv:2512.16229*, 2025. URL <https://arxiv.org/abs/2512.16229>.
- Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7b: Diffusion large language models. *arXiv preprint arXiv:2508.15487*, 2025. URL <https://arxiv.org/abs/2508.15487>.

A Theoretical View of MultiTF

This appendix provides a simple theoretical view of Multi-block Teacher Forcing (MultiTF). The goal is not to prove that MultiTF directly improves downstream accuracy. Instead, we show that MultiTF can be interpreted as a coverage-based surrogate for the ideal MultiBD training objective, and that its approximation gap is controlled by the mismatch in *running-set* coverage and noise-ratio distributions.

Ideal MultiBD objective. Let $\mathcal{R} = \{a, \dots, c\}$ denote a consecutive MultiBD *running-set* with size $|\mathcal{R}| \leq G_{\max}$, where $G_{\max}$ is the maximum *noise-group* size used in MultiTF. For a clean sequence $\mathbf{x}_0$, noise ratios $\mathbf{t}$, and model θ , define the state loss

$$\ell_{\theta}(\mathbf{x}_0, \mathcal{R}, \mathbf{t}) = -\frac{1}{|\mathcal{M}_{\mathcal{R}}|} \sum_{i \in \mathcal{M}_{\mathcal{R}}} \log p_{\theta}(x_0^i | \mathbf{x}_0^{(<a)}, \mathbf{b}_{\mathcal{R}, \mathbf{t}}), \quad (\text{A.1})$$

where $\mathbf{x}_0^{(<a)}$ is the clean prefix before $\mathcal{R}$, $\mathbf{b}_{\mathcal{R}, \mathbf{t}}$ denotes the noisy blocks inside $\mathcal{R}$, and $\mathcal{M}_{\mathcal{R}}$ denotes masked positions in the *running-set*.

Let $p_{\inf}(\mathcal{R}, \mathbf{t})$ be the inference-time distribution of MultiBD states, and let $q_{\text{MultiTF}}(\mathcal{R}, \mathbf{t})$ be the training-state distribution induced by MultiTF *group-layouts* and the chain-uniform *noise-scheduler*. The ideal MultiBD objective is

$$\mathcal{L}_{\text{MultiBD}}^*(\theta) = \mathbb{E}_{\mathbf{x}_0} \mathbb{E}_{(\mathcal{R}, \mathbf{t}) \sim p_{\inf}} [\ell_{\theta}(\mathbf{x}_0, \mathcal{R}, \mathbf{t})], \quad (\text{A.2})$$

while MultiTF minimizes the surrogate objective

$$\mathcal{L}_{\text{MultiTF}}(\theta) = \mathbb{E}_{\mathbf{x}_0} \mathbb{E}_{(\mathcal{R}, \mathbf{t}) \sim q_{\text{MultiTF}}} [\ell_{\theta}(\mathbf{x}_0, \mathcal{R}, \mathbf{t})]. \quad (\text{A.3})$$

Systematic shifts cover bounded running-sets. Assume the sequence is padded so that boundary effects can be ignored. For a fixed *noise-group* size $g \in \{2, \dots, G_{\max}\}$, MultiTF constructs g shifted layouts. Then every consecutive *running-set* $\mathcal{R} = \{a, \dots, a + g - 1\}$ appears as one *noise-group* in exactly one shifted layout for that g .

Proof. For a fixed g , each shifted layout places group boundaries every g blocks with a different offset. For a *running-set* starting at block a , choosing the shift $h = (a - 1) \bmod g$ aligns a group boundary with a , so $\{a, \dots, a + g - 1\}$ appears as one *noise-group*. The shift is unique modulo g , so the *running-set* appears once among the g shifted layouts.

Thus, systematic layouts cover all consecutive *running-sets* with size between 2 and $G_{\max}$. Equivalently, for each fixed g , every block appears once at every group-relative logical slot across the g shifts. Random layouts do not change this support guarantee, but add additional samples with non-regular *noise-group-size* combinations.

Objective mismatch bound. We next bound the gap between the ideal MultiBD objective and the MultiTF surrogate objective. Let $p_{\mathcal{R}}$ and $q_{\mathcal{R}}$ be the marginal distributions over *running-sets* under $p_{\inf}$ and q_{MultiTF} , respectively.

We assume:

A1. Bounded MultiBD states. The inference distribution $p_{\inf}$ is supported on consecutive *running-sets* with $2 \leq |\mathcal{R}| \leq G_{\max}$.

A2. Bounded loss. For all $\theta, \mathbf{x}_0, \mathcal{R}, \mathbf{t}$,

$$0 \leq \ell_{\theta}(\mathbf{x}_0, \mathcal{R}, \mathbf{t}) \leq M.$$

A3. Lipschitz dependence on noise ratios. For every $\theta, \mathbf{x}_0, \mathcal{R}$, the state loss is L_t -Lipschitz in the noise-ratio vector:

$$|\ell_{\theta}(\mathbf{x}_0, \mathcal{R}, \mathbf{t}) - \ell_{\theta}(\mathbf{x}_0, \mathcal{R}, \mathbf{t}')| \leq L_t \|\mathbf{t} - \mathbf{t}'\|_1. \quad (\text{A.4})$$

Here $\text{TV}(p, q) = \frac{1}{2} \sum_x |p(x) - q(x)|$ denotes the total variation distance between two discrete distributions.

Define the *running-set* distribution mismatch as

$$\delta_{\mathcal{R}} = \text{TV}(p_{\mathcal{R}}, q_{\mathcal{R}}),$$

and assume the conditional noise-ratio mismatch satisfies

$$W_1(p_{\inf}(\mathbf{t} \mid \mathcal{R}), q_{\text{MultiTF}}(\mathbf{t} \mid \mathcal{R})) \leq \delta_t$$

for every *running-set* $\mathcal{R}$, where W_1 is the Wasserstein-1 distance under the ℓ_1 metric.

Under these assumptions, for any model θ ,

$$|\mathcal{L}_{\text{MultiBD}}^*(\theta) - \mathcal{L}_{\text{MultiTF}}(\theta)| \leq M\delta_{\mathcal{R}} + L_t\delta_t. \quad (\text{A.5})$$

Proof. For clarity, omit the outer expectation over $\mathbf{x}_0$. We decompose the objective gap into a *running-set* distribution term and a conditional noise-distribution term:

$$|\mathbb{E}_{p_{\mathcal{R}}p(\mathbf{t}|\mathcal{R})}[\ell_{\theta}] - \mathbb{E}_{q_{\mathcal{R}}q(\mathbf{t}|\mathcal{R})}[\ell_{\theta}]| \leq |\mathbb{E}_{p_{\mathcal{R}}p(\mathbf{t}|\mathcal{R})}[\ell_{\theta}] - \mathbb{E}_{q_{\mathcal{R}}p(\mathbf{t}|\mathcal{R})}[\ell_{\theta}]| + |\mathbb{E}_{q_{\mathcal{R}}p(\mathbf{t}|\mathcal{R})}[\ell_{\theta}] - \mathbb{E}_{q_{\mathcal{R}}q(\mathbf{t}|\mathcal{R})}[\ell_{\theta}]|. \quad (\text{A.6})$$

The first term is bounded by $M\text{TV}(p_{\mathcal{R}}, q_{\mathcal{R}}) = M\delta_{\mathcal{R}}$, since the loss is bounded in $[0, M]$. The second term is bounded by $L_t\delta_t$ by the Lipschitz assumption and the definition of W_1 . Combining the two terms gives Eq. A.5.

Excess target risk. Let $\hat{\theta}$ be a model whose MultiTF objective is within ϵ_{opt} of the best model in a hypothesis class Θ :

$$\mathcal{L}_{\text{MultiTF}}(\hat{\theta}) \leq \min_{\theta \in \Theta} \mathcal{L}_{\text{MultiTF}}(\theta) + \epsilon_{\text{opt}}.$$

Then

$$\mathcal{L}_{\text{MultiBD}}^*(\hat{\theta}) - \min_{\theta \in \Theta} \mathcal{L}_{\text{MultiBD}}^*(\theta) \leq 2(M\delta_{\mathcal{R}} + L_t\delta_t) + \epsilon_{\text{opt}}. \quad (\text{A.7})$$

This bound shows that reducing *running-set* distribution mismatch $\delta_{\mathcal{R}}$ and noise-ratio mismatch δ_t directly tightens the gap between MultiTF training and ideal MultiBD inference. Systematic shifts reduce support mismatch by covering bounded consecutive *running-sets* up to size G_{max} , random layouts add distributional diversity, and the chain-uniform *noise-scheduler* reduces noise-ratio mismatch by producing heterogeneous slot-wise noise gaps. Therefore, MultiTF can be viewed as a coverage-based surrogate for the ideal MBD-LM objective.

B MultiTF Training Implementation Details

This appendix provides implementation details for Multi-block Teacher Forcing (MultiTF), which post-trains BD-LMs into MBD-LMs. The terminology follows Section 3.2: training-side structures are called *noise-groups*, *group-layouts*, and *noise-schedulers*, while inference-side structures are called *Block Buffers* and *slots*. We use G_{max} for the maximum *noise-group* size, Λ for the set of *group-layouts*, λ for one *group-layout*, and H_m for one *noise-group*. We use VeOmni (Ma et al., 2025) as the training framework. SDAR models are post-trained on reasoning/code data from prior studies (Boizard et al., 2025; jtatman, 2025); LLaDA2.x and DMax-enhanced models are post-trained on the corresponding reasoning/code mixtures used by their base recipes.

B.1 Group-Layout Construction

MultiTF constructs a *group-layout set*

$$\Lambda = \Lambda_{\text{sys}} \cup \Lambda_{\text{rand}},$$

where Λ_{sys} contains systematic shifted layouts and Λ_{rand} contains random layouts. Each *group-layout* $\lambda = (H_1, \dots, H_{|\lambda|})$ partitions the block sequence $[\mathbf{b}_1, \dots, \mathbf{b}_K]$ into consecutive *noise-groups*. Each *noise-group* $H_m = \{a_m, \dots, c_m\}$ has the same consecutive-block form as a possible MultiBD *running-set*.

Systematic layouts. For each *noise-group* size $g \in \{2, \dots, G_{\max}\}$ and shift $h \in \{0, \dots, g-1\}$, MultiTF constructs a shifted layout $\lambda_{g,h}$ by placing group boundaries every g blocks with offset h . Formally, define the boundary set

$$\mathcal{B}_{g,h} = \text{sort}\left(\{1, K+1\} \cup \{1+h+qg : q \in \mathbb{Z}, 1 < 1+h+qg < K+1\}\right).$$

Let $\mathcal{B}_{g,h} = (r_1, \dots, r_{n_{g,h}+1})$ after sorting. The q -th *noise-group* in $\lambda_{g,h}$ is

$$H_{g,h,q} = \{r_q, \dots, r_{q+1}-1\}, \quad q = 1, \dots, n_{g,h}.$$

Boundary *noise-groups* can be shorter than g , while interior *noise-groups* have size g . The systematic layout set is

$$\Lambda_{\text{sys}} = \{\lambda_{g,h} : g \in \{2, \dots, G_{\max}\}, h \in \{0, \dots, g-1\}\}.$$

Ignoring boundary effects, every consecutive *running-set* $\{a, \dots, a+g-1\}$ of length g appears as one *noise-group* in exactly one shifted layout by choosing $h = (a-1) \bmod g$. Equivalently, for each fixed g , every block appears once at every group-relative position across the g shifts. The number of systematic layouts is therefore

$$|\Lambda_{\text{sys}}| = \sum_{g=2}^{G_{\max}} g = \frac{(G_{\max}+2)(G_{\max}-1)}{2}. \quad (\text{B.1})$$

Random layouts. Systematic layouts provide structured coverage but are regular by construction. To increase layout diversity, MultiTF further samples N_{rand} random layouts. For each random layout, we sequentially draw group sizes

$$g_m \sim \text{Uniform}\{2, \dots, G_{\max}\}$$

and form consecutive groups

$$H_m = \{a_m, \dots, \min(a_m + g_m - 1, K)\}, \quad a_{m+1} = \min(a_m + g_m, K+1),$$

until the full block sequence is covered. These random layouts add non-regular *noise-group-size* combinations and boundary patterns without replacing the coverage guarantee of systematic layouts. The total number of layout variants per clean sequence is

$$|\Lambda| = \frac{(G_{\max}+2)(G_{\max}-1)}{2} + N_{\text{rand}}. \quad (\text{B.2})$$

All layouts are batched as independent input sequences during post-training. This increases the effective number of training states per clean sample, but also increases training cost; exact settings are reported in Table 5. A theoretical coverage view is provided in Appendix A.

B.2 Chain-uniform Noise-Scheduler

For each *noise-group* $H_m = (j_1, \dots, j_{n_m})$, MultiTF applies the chain-uniform *noise-scheduler* used in Algorithm 1. We first define an effective upper bound

$$t_{\text{eff}} = t_{\text{high}} - \rho(t_{\text{high}} - t_{\text{low}}), \quad (\text{B.3})$$

where ρ is the noise-transition margin ratio, corresponding to **noise_transition_margin_ratio** in the implementation. This parameter is independent of the random *noise-scheduler* power-law bias γ_{rand} , which is used only for the random *noise-scheduler* ablation.

For each group, a group-level floor ℓ is first sampled from the lower part of the noise range. Then each block samples its mask ratio from the interval between the current floor and the effective upper bound, and the sampled ratio becomes the floor for the next block:

$$\ell \sim \mathcal{U}(t_{\text{low}}, t_{\text{eff}}), \quad t_{j_i} \sim \mathcal{U}(\ell, t_{\text{eff}}), \quad \ell \leftarrow t_{j_i}, \quad i = 1, \dots, n_m. \quad (\text{B.4})$$

This construction produces monotonic but randomized slot-wise mask ratios inside each *noise-group*. Compared with the fixed-step D2F schedule over a long noisy sequence, the resulting groups have larger and more variable block-level noise-ratio gaps, matching the heterogeneous active blocks observed during MultiBD inference.

For each block with mask ratio t_{j_i} , MultiTF replaces $\lfloor B \cdot t_{j_i} \rfloor$ randomly selected token positions in $\mathbf{b}_{j_i}$ with $[\mathbf{M}]$. For a layout λ , the resulting noisy sequence is denoted as $\mathbf{x}_t^\lambda$.

B.3 Group-Aware Dual-Stream Mask

Following the TF-style construction of Block Diffusion, MultiTF concatenates the noisy and clean sequences into the input sequence

$$\mathbf{X}_\lambda = [\mathbf{x}_t^\lambda; \mathbf{x}_0]. \quad (\text{B.5})$$

The attention mask has the block form

$$\mathbf{A}_\lambda = \begin{bmatrix} \mathbf{M}_{\text{GD}} & \mathbf{M}_{\text{GOC}} \\ 0 & \mathbf{M}_{\text{BC}} \end{bmatrix}, \quad (\text{B.6})$$

where $\mathbf{M}_{\text{GD}}$ is the group-aware diagonal mask on the noisy part, $\mathbf{M}_{\text{GOC}}$ is the group-aware offset-causal mask from noisy tokens to clean tokens, and $\mathbf{M}_{\text{BC}}$ is the standard block-causal mask on the clean part.

Let $\mathcal{N}_\lambda$ and $\mathcal{C}$ denote token positions in the noisy and clean parts, respectively. Let $g(i)$ be the *noise-group* index of token i , $\beta(i)$ be its block index, and $\alpha(i)$ be the first block index of the *noise-group* containing i . The three masks are defined as

$$[\mathbf{M}_{\text{GD}}]_{ij} = 1 \iff i, j \in \mathcal{N}_\lambda, \quad g(i) = g(j), \quad \beta(j) \leq \beta(i), \quad (\text{B.7})$$

$$[\mathbf{M}_{\text{GOC}}]_{ij} = 1 \iff i \in \mathcal{N}_\lambda, \quad j \in \mathcal{C}, \quad \beta(j) < \alpha(i), \quad (\text{B.8})$$

$$[\mathbf{M}_{\text{BC}}]_{ij} = 1 \iff i, j \in \mathcal{C}, \quad \beta(j) \leq \beta(i), \quad (\text{B.9})$$

and all other entries are zero. Thus, noisy tokens can attend to same-*noise-group* noisy tokens from the same or preceding blocks, each *noise-group* can condition on clean prefix blocks before it, and clean tokens never attend to noisy tokens. This implements the visibility pattern required by Equation 3.1 without information leakage.

B.4 MultiTF Objective and Model-specific Training Recipes

MultiTF defines the training-state construction: the layout λ , the noisy sequence $\mathbf{x}_t^\lambda$, the clean sequence $\mathbf{x}_0$, and the *Group-Aware Dual-Stream Mask* $\mathbf{A}_\lambda$. Different base BD-LMs can reuse the same MultiTF input sequences while keeping their own model-specific training recipes.

B.4.1 Default MultiTF CE Objective

The default MultiTF objective is masked-token cross-entropy on masked positions in the noisy part of $\mathbf{X}_\lambda$. Let

$$\mathcal{M}_\lambda = \{i : \mathbf{x}_t^\lambda[i] = [\mathbf{M}]\} \quad (\text{B.10})$$

denote the masked positions. The objective is

$$\mathcal{L}_{\text{MultiTF}}(\theta) = -\mathbb{E}_{\lambda, \mathbf{t}, \mathbf{x}_0} \left[\frac{1}{|\mathcal{M}_\lambda|} \sum_{i \in \mathcal{M}_\lambda} \log p_\theta(x_0^i | \mathbf{X}_\lambda, \mathbf{A}_\lambda) \right]. \quad (\text{B.11})$$

This objective is used for BD-LMs whose original training recipe is standard masked-token CE.

Algorithm 2 DMax OPUT Self-Denoising Branch

Require: Model θ ; input sequence $\mathbf{X}_\lambda = [\mathbf{x}_t^\lambda; \mathbf{x}_0]$; noisy length N ; mask token id m .
 1: Run a no-gradient forward pass on the noisy part: $\mathbf{L} \leftarrow \theta(\mathbf{X}_\lambda)_{:N}$.
 2: Compute argmax predictions $\hat{\mathbf{x}} \leftarrow \arg \max \mathbf{L}$.
 3: Replace masked positions in $\mathbf{x}_t^\lambda$ with $\hat{\mathbf{x}}$.
 4: **return** the partially self-denoised input sequence.

B.4.2 DMax-enhanced Models: OPUT Self-denoising

For DMax-enhanced models, we keep the same MultiTF input sequences and add the DMax OPUT self-denoising branch. For each MultiTF input sequence, OPUT forms two branches. The standard branch computes the training loss on the original noisy input sequence. The self-denoising branch first runs a no-gradient forward pass, replaces masked positions in the noisy part with the model’s argmax predictions, and then computes the loss on this partially self-denoised input. Gradients flow only through the second forward pass of the self-denoising branch. This exposes the model to partially self-generated states while keeping the MultiTF layout and attention-mask construction unchanged. The procedure is summarized in Algorithm 2.

B.4.3 SDAR Models: Block-wise Noise-weighted CE

For SDAR models, we also reuse the same MultiTF input sequences and *Group-Aware Dual-Stream Masks*. The difference lies in the loss normalization. Instead of computing one global masked-token CE over all masked positions, SDAR applies a block-wise noise-weighted CE, where the loss of each block is normalized by the mask ratio applied to that block.

Let $\mathcal{B}_k$ denote token positions of block k in the noisy part, and let

$$\mathcal{M}_{\lambda,k} = \mathcal{M}_\lambda \cap \mathcal{B}_k \quad (\text{B.12})$$

be the masked positions in block k under layout λ . Let $t_{\lambda,k}$ denote the mask ratio assigned to block k . The SDAR-style MultiTF objective is

$$\mathcal{L}_{\text{MultiTF}}^{\text{SDAR}}(\theta) = -\mathbb{E}_{\lambda, \mathbf{t}, \mathbf{x}_0} \left[\frac{1}{K} \sum_{k=1}^K \frac{1}{\max(t_{\lambda,k}, \epsilon)} \sum_{i \in \mathcal{M}_{\lambda,k}} \log p_\theta(x_0^i | \mathbf{X}_\lambda, \mathbf{A}_\lambda) \right], \quad (\text{B.13})$$

where ϵ is a small constant used for numerical stability. This block-wise normalization extends the diffusion loss to the full block sequence while preserving the per-block noise weighting used by SDAR. It differs from Equation B.11, which normalizes the loss globally over all masked positions in the noisy part.

B.5 Sorted-uniform Scheduler Baseline

The sorted-uniform *noise-scheduler* is a baseline for constructing monotonic block-level noise within each *noise-group*. For a *noise-group* $H_m = (j_1, \dots, j_{n_m})$, it independently samples n_m mask ratios from a uniform distribution and then sorts them in ascending order before assigning them to the blocks in the *noise-group*, as summarized in Algorithm 3:

$$u_1, \dots, u_{n_m} \stackrel{\text{i.i.d.}}{\sim} \mathcal{U}(t_{\text{low}}, t_{\text{high}}), \quad u_{(1)} \leq \dots \leq u_{(n_m)}, \\ t_{j_i} = u_{(i)}, \quad i = 1, \dots, n_m.$$

This produces a monotonic noise pattern similar in spirit to D2F. However, unlike the chain-uniform *noise-scheduler* in Appendix B.2, the gaps between adjacent slots are only induced by order statistics of uniformly sampled values and are not explicitly encouraged to be large.

C MultiBD Inference Implementation Details

This appendix expands the optimized MultiBD inference algorithm introduced in Section 3.3. The main design goal is to execute the MultiBD *running-set* in Equation 3.1 with a static physical input shape, while preserving prefix KV-cache reuse.

Algorithm 3 Sorted-uniform Block-level Noise-Scheduler

Require: Noise-group $H_m = (j_1, \dots, j_{n_m})$; noise bounds $t_{\text{low}}, t_{\text{high}}$.

- 1: **for** $i \leftarrow 1$ **to** n_m **do**
- 2: Sample $u_i \sim \mathcal{U}(t_{\text{low}}, t_{\text{high}})$.
- 3: **end for**
- 4: Sort sampled ratios: $u_{(1)} \leq \dots \leq u_{(n_m)}$.
- 5: **for** $i \leftarrow 1$ **to** n_m **do**
- 6: Assign $t_{j_i} \leftarrow u_{(i)}$.
- 7: **end for**
- 8: **return** block-level mask ratios $\{t_{j_i}\}_{i=1}^{n_m}$.

Algorithm 4 Naive MultiBD with a Dynamic Running-Set

Require: Model θ ; block size B ; thresholds $\tau_{\text{add}}, \tau_{\text{semi}}, \tau_{\text{M2T}}$.

// **Initialize dynamic MultiBD state**

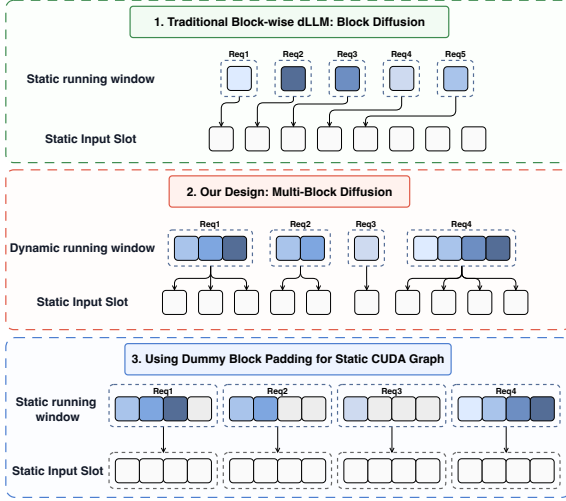
- 1: Initialize prefix KV cache $\mathcal{K} \leftarrow \emptyset$ and dynamic *running-set* $\mathcal{Y} \leftarrow \emptyset$.
- 2: Append one fully masked ACTIVE block to $\mathcal{Y}$.
- 3: **while** generation is not complete **do**
- 4: // **Grow the running-set dynamically**
- 5: **if** the latest active block has progress $> \tau_{\text{add}}$ and EOS has not appeared **then**
- 6: Append a fully masked future block to $\mathcal{Y}$.
- 7: **end if**
- 8: // **Decode all blocks in the current running-set**
- 9: Run θ on $\mathcal{Y}$ with prefix cache $\mathcal{K}$.
- 10: **for** each active block $b \in \mathcal{Y}$ **do**
- 11: Accept masked positions with confidence $> \tau_{\text{M2T}}$.
- 12: **if** the previous active block is semi-complete and no token is accepted **then**
- 13: Accept the highest-confidence masked position.
- 14: **end if**
- 15: **if** b is fully decoded **then**
- 16: Mark b as TO-CACHE.
- 17: **end if**
- 18: **end for**
- 19: // **Commit completed prefix blocks**
- 20: **while** the front block of $\mathcal{Y}$ is TO-CACHE **do**
- 21: Write the front block into $\mathcal{K}$ and remove it from $\mathcal{Y}$.
- 22: **end while**
- 23: **end while**
- 24: **return** generated tokens.

C.1 A dynamic running-set prevents static-shape execution.

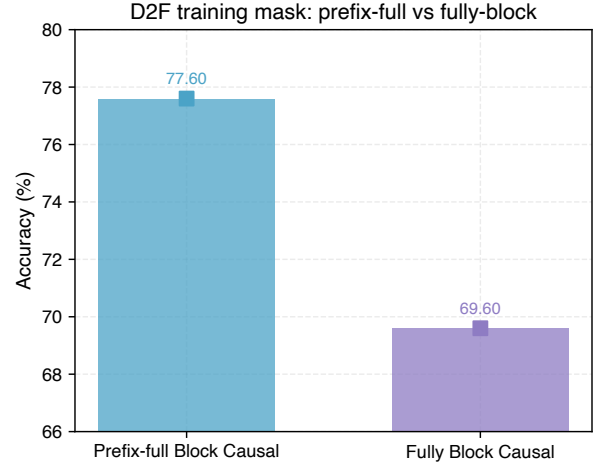
A direct implementation of MultiBD maintains a dynamic *running-set* in addition to the committed prefix cache. When the latest active block reaches an add-block threshold, the decoder appends a fully masked future block to the *running-set*. When the front active block is completed, the decoder writes it into the KV cache and removes it from the *running-set*. This dynamic procedure exposes inter-block parallelism, but the number of active tokens changes across decoding steps and across requests. As shown in Figure 6a(2), such shape variation is unfriendly to CUDA Graph capture and replay.

C.2 A fixed Block Buffer implements MultiBD states.

Optimized MultiBD replaces dynamic appending with a fixed-size *Block Buffer*. The *Block Buffer* contains N_{buf} physical block slots. At each decoding step, active slots represent the logical *running-set* $\mathcal{R}_s$, while dummy slots reserve capacity for future blocks. Adding a future block therefore activates an existing dummy slot instead of extending the physical input sequence. When the front active block is completed, it is committed to the KV cache, removed from the *running-set*, and the *Block Buffer* slides forward by replacing the consumed slot with a new dummy slot at the tail. This realizes MultiBD while keeping the number of processed buffer tokens fixed at $N_{\text{buf}} \cdot B$.



(a) CUDA Graph compatibility across decoding designs. (1) SingleBD uses a fixed single active block but exposes no inter-block parallelism. (2) Naive MultiBD appends future blocks dynamically, making the *running-set* length change over time. (3) Optimized MultiBD maps the logical *running-set* into a fixed-size *Block Buffer* with dummy slots, keeping tensor shapes static for CUDA Graph capture and replay.



(b) Making D2F fully block-causal hurts accuracy. Prefix-full attention gives D2F stronger noisy-prefix visibility but is not naturally compatible with prefix KV caching. Directly replacing it with a fully block-causal mask improves cache compatibility but drops accuracy from 77.60% to 69.60%.

Figure 6 Static-shape execution and prefix-cache compatibility analyses. Left: optimized MultiBD keeps tensor shapes static through a fixed-size *Block Buffer*, enabling CUDA Graph capture and replay. Right: making D2F fully block-causal improves cache compatibility but substantially hurts accuracy.

C.3 Block states advance the fixed Block Buffer.

Each physical slot in the *Block Buffer* follows the transition

$$\text{DUMMY} \rightarrow \text{ACTIVE} \rightarrow \text{TO-CACHE} \rightarrow \text{IN-CACHE}.$$

A DUMMY slot is an idle placeholder that preserves the static buffer shape. An ACTIVE slot participates in the current MultiBD forward pass. A TO-CACHE block has completed decoding and is ready to be committed. An IN-CACHE block has been written into the prefix KV cache and no longer belongs to the active part of the *running-set*. These state transitions implement the logical evolution of $\mathcal{R}_s$ without changing the physical input shape.

C.4 Thresholds control activation and token updates.

MultiBD uses separate thresholds for block activation, fallback progress, and token updates. The add-block threshold τ_{add} controls when a future block can enter the fixed *Block Buffer*. The stability threshold τ_{stable} prevents premature activation when the current latest active block is still unstable. The semi-completion threshold τ_{semi} allows later active blocks to use the top-1 context of a preceding block once it has made sufficient progress, even before it is fully cached. The M2T threshold τ_{M2T} controls mask-to-token acceptance, and the optional T2T threshold τ_{T2T} controls token-to-token revision for models that support T2T updates.

This separation is important because M2T and T2T updates have different reliability profiles. M2T introduces new content into an active block, while T2T overwrites tentative content before commitment. Using separate thresholds stabilizes concurrent block refinement and reduces error propagation across the *running-set*.

Algorithm 5 Optimized MultiBD with a Fixed *Block Buffer*

Require: Model θ ; block size B ; buffer size N_{buf} ; thresholds τ_{add} , τ_{semi} , τ_{stable} , τ_{M2T} , and optional τ_{T2T} .

```

1: // Initialize fixed Block Buffer
2: Initialize prefix KV cache  $\mathcal{K}$  and a fixed Block Buffer  $\mathcal{W}$  with  $N_{\text{buf}}$  slots.
3: Set  $\mathcal{W}[0]$  to a fully masked ACTIVE block and all remaining slots to DUMMY.
4: while generation is not complete do
5:   // Activate future blocks without changing shape
6:   Let  $\mathcal{R}$  be the non-DUMMY resident blocks in  $\mathcal{W}$ .
7:   Let  $b_{\text{last}}$  be the last ACTIVE block in  $\mathcal{R}$ .
8:   if  $b_{\text{last}}$  satisfies progress  $> \tau_{\text{add}}$  and stability  $> \tau_{\text{stable}}$  then
9:     Activate the first trailing DUMMY slot if one exists.
10:  end if
11:  // Decode the static Block Buffer
12:  Run  $\theta$  on the static  $N_{\text{buf}} \cdot B$  Block Buffer tokens with prefix cache  $\mathcal{K}$ .
13:  for each ACTIVE block  $b \in \mathcal{W}$  do
14:    Accept masked positions with confidence  $> \tau_{\text{M2T}}$ .
15:    if no masked position is accepted and the preceding active block is semi-complete then
16:      Accept the highest-confidence masked position in  $b$ .
17:    end if
18:    if T2T revision is enabled then
19:      Revise eligible filled but uncommitted positions with confidence  $> \tau_{\text{T2T}}$ .
20:    end if
21:    if  $b$  is complete and all preceding resident blocks are cached or ready-to-cache then
22:      Mark  $b$  as TO-CACHE.
23:    end if
24:  end for
25:  // Commit prefix blocks and slide the buffer
26:  while the front slot of  $\mathcal{W}$  is TO-CACHE do
27:    Write the front block into  $\mathcal{K}$ ; its state becomes IN-CACHE.
28:    Pop the front slot and append a new DUMMY slot at the tail.
29:  end while
30: end while
31: return generated tokens.

```

C.5 Prefix Caching and Fully Block-Causal D2F

Native D2F is not directly prefix-cache compatible. Prefix caching is a key advantage of BD-LMs. In SingleBD, completed blocks form an immutable clean prefix, so their KV states can be stored and directly reused in later steps. As shown in Figure 7(1), only the current noisy block requires repeated computation. By contrast, native D2F uses prefix-full attention. Future noisy blocks condition on a prefix-full context, where prefix states are not organized as immutable block-causal prefix pages in the standard BD-LM cache. As illustrated in Figure 7(2), their KV states cannot be reused in the same way as SingleBD prefix blocks.

Fully block-causal D2F variant. To isolate the prefix-caching issue, we construct a fully block-causal D2F variant. Let the full clean sequence be partitioned into BD-LM blocks:

$$\mathbf{x}_0 = [\mathbf{b}_1, \dots, \mathbf{b}_K], \quad \mathbf{b}_k \in \mathcal{V}^B.$$

Suppose native D2F uses a token-level clean prefix

$$\mathbf{x}_0^{\text{pre}} = (x_0^1, \dots, x_0^P),$$

where P can be arbitrary and need not be divisible by B . Let

$$a = \left\lfloor \frac{P}{B} \right\rfloor + 1, \quad r = P - (a - 1)B$$

denote the first block that contains suffix tokens and the number of prefix tokens inside this boundary block, respectively. Then $\mathbf{b}_1, \dots, \mathbf{b}_{a-1}$ are complete clean prefix blocks, while $\mathbf{b}_a$ may contain both prefix tokens and suffix tokens.

We use $\mathbf{b}_a$ as the first noisy block of the D2F-style suffix, rather than inserting padding tokens. For the boundary block, only its suffix positions are noised and included in the loss:

$$\mathcal{I}_a = \{r + 1, \dots, B\}.$$

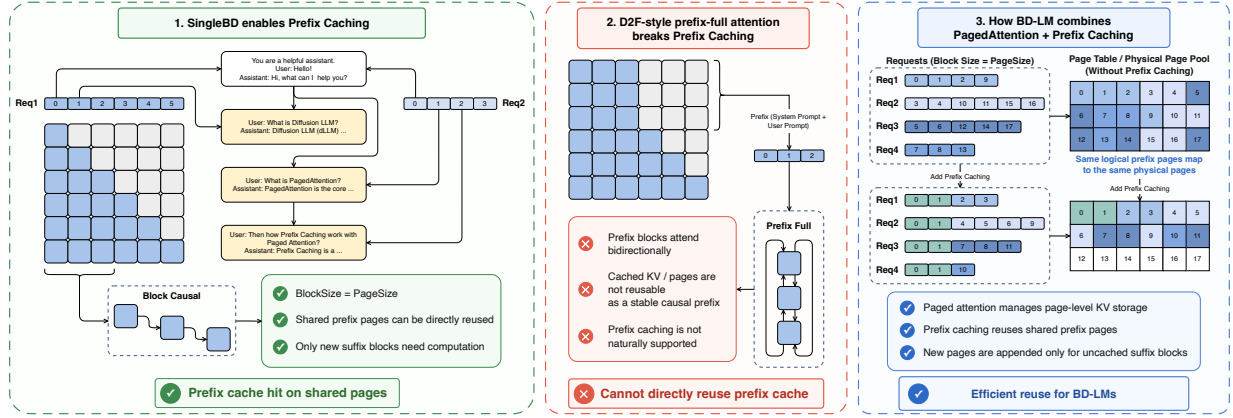


Figure 7 Prefix caching in block-causal BD-LMs. (1) SingleBD keeps completed blocks as an immutable clean prefix, enabling direct KV-cache reuse. (2) D2F-style prefix-full attention breaks this cache semantics because noisy prefix blocks are not reusable as stable causal prefix pages. (3) *Block Buffer* MultiBD separates cached prefix blocks from active *Block Buffer* slots, enabling prefix KV reuse while refining multiple active blocks.

For later blocks $j > a$, all positions belong to the suffix:

$$\mathcal{I}_j = \{1, \dots, B\}.$$

We then apply a monotonic D2F-style *noise-scheduler* to the valid suffix positions of blocks $a, \dots, K$:

$$0 \leq t_a < t_{a+1} < \dots < t_K \leq 1.$$

Let $\bar{\mathbf{b}}_{j,t_j}$ denote the partially corrupted block, where positions in $\mathcal{I}_j$ are corrupted by $q_{t_j}(\cdot | \mathbf{b}_j)$ and positions outside $\mathcal{I}_j$ are kept clean. For the boundary block, this means that the prefix part of $\mathbf{b}_a$ remains clean, while the suffix part is noised.

The fully block-causal D2F variant factorizes the suffix as

$$p_\theta(\mathbf{x}_0^{\text{sup}} | \mathbf{x}_0^{(<a)}, \bar{\mathbf{b}}_{a,t_a}, \dots, \bar{\mathbf{b}}_{K,t_K}) = \prod_{j=a}^K p_\theta(\mathbf{b}_j^{\mathcal{I}_j} | \mathbf{x}_0^{(<a)}, \bar{\mathbf{b}}_{a,t_a}, \dots, \bar{\mathbf{b}}_{j,t_j}), \quad (\text{C.1})$$

where $\mathbf{x}_0^{(<a)} = [\mathbf{b}_1, \dots, \mathbf{b}_{a-1}]$ is the block-causal clean prefix and $\mathbf{b}_j^{\mathcal{I}_j}$ denotes the suffix positions of block j . The loss is computed only on masked positions within $\mathcal{I}_j$.

Compared with native D2F, this variant changes the training-state construction by replacing prefix-full attention with a fully block-causal attention. Equivalently, it completes the arbitrary token-level prefix to the next block boundary using real continuation tokens from the training sequence, and treats the boundary block as the first block in the noisy suffix. This construction is the training-side counterpart of the extreme MBD-LM state discussed in Section 3.1, where the *running-set* covers all suffix blocks and follows a monotonic D2F-style *noise-scheduler*.

Fully block-causal D2F is not a sufficient fix. Although the fully block-causal variant improves cache compatibility, it substantially hurts accuracy. As shown in Figure 6b, changing D2F from prefix-full attention to fully block-causal attention drops accuracy from 77.60% to 69.60%. This suggests that D2F relies on stronger prefix-full visibility, and cache compatibility cannot be obtained by simply restricting the attention mask. This result further motivates MultiTF, which keeps the block-causal cached-prefix interface while training on bounded *noise-groups* that better match MultiBD inference.

Block Buffer MultiBD preserves cache semantics. Our *Block Buffer* MultiBD design preserves prefix caching by construction. Committed blocks become immutable IN-CACHE prefix context and are represented only through cached KV states. Active blocks remain inside the *Block Buffer* and are recomputed during iterative

Table 4 Inference hyperparameters for all evaluated configurations. τ_{add} controls when a future block is activated; τ_{semi} controls semi-completion or fallback progress; τ_{stable} controls activation stability; τ_{M2T} and τ_{T2T} are confidence thresholds for mask-to-token filling and token-to-token revision.

Configuration	Task	Buffer	Block	Max Len	Max New	Max NFE	τ_{add}	τ_{semi}	τ_{stable}	τ_{M2T}	τ_{T2T}
LLaDA2-Mini-DMax											
SingleBD (Native)	Math	1	32	4096	4096	1024	—	—	—	0.50	—
SingleBD (Native)	Code	1	32	4096	4096	1024	—	—	—	0.65	—
MultiBD (training-free)	Math	2	32	4096	4096	1024	0.10	0.90	0.50	0.50	—
MultiBD (training-free)	Code	2	32	4096	4096	1024	0.90	0.90	0.50	0.65	—
MBD-LLaDA2-Mini-DMax	Math	2	32	4096	4096	1024	0.10	0.90	0.50	0.50	—
MBD-LLaDA2-Mini-DMax	Code	2	32	4096	4096	1024	0.90	0.90	0.50	0.65	—
LLaDA2-Mini											
SingleBD (Native)	Math	1	32	4096	4096	1024	—	—	—	0.95	—
SingleBD (Native)	Code	1	32	4096	4096	1024	—	—	—	0.95	—
MultiBD (training-free)	Math	2	32	4096	4096	1024	0.10	0.90	—	0.95	—
MultiBD (training-free)	Code	2	32	4096	4096	1024	0.90	0.90	—	0.95	—
MBD-LLaDA2-Mini	Math	2	32	4096	4096	1024	0.10	0.90	—	0.95	—
MBD-LLaDA2-Mini	Code	2	32	4096	4096	1024	0.90	0.90	—	0.95	—
SDAR-8B-Chat-b32											
SingleBD (Native)	Math	1	32	4096	4096	1024	—	—	—	0.95	—
SingleBD (Native)	Code	1	32	4096	4096	1024	—	—	—	0.95	—
MultiBD (training-free)	Math	4	32	4096	4096	1024	0.10	0.90	—	0.95	—
MultiBD (training-free)	Code	4	32	4096	4096	1024	0.90	0.90	—	0.95	—
MBD-SDAR-8B-Chat-b32	Math	4	32	4096	4096	1024	0.10	0.90	—	0.95	—
MBD-SDAR-8B-Chat-b32	Code	4	32	4096	4096	1024	0.90	0.90	—	0.95	—
SDAR-8B-Chat-b4											
SingleBD (Native)	Math	1	4	4096	4096	1024	—	—	—	0.95	—
SingleBD (Native)	Code	1	4	4096	4096	1024	—	—	—	0.95	—
MultiBD (training-free)	Math	4	4	4096	4096	1024	0.10	0.25	—	0.95	—
MultiBD (training-free)	Code	4	4	4096	4096	1024	0.75	0.75	—	0.95	—
MBD-SDAR-8B-Chat-b4	Math	4	4	4096	4096	1024	0.10	0.25	—	0.95	—
MBD-SDAR-8B-Chat-b4	Code	4	4	4096	4096	1024	0.75	0.75	—	0.95	—
LLaDA2-Mini-CAP											
SingleBD (Native)	Math	1	32	4096	4096	1024	—	—	—	0.95	—
SingleBD (Native)	Code	1	32	4096	4096	1024	—	—	—	0.95	—
MultiBD (training-free)	Math	2	32	4096	4096	1024	0.10	0.90	—	0.95	—
MultiBD (training-free)	Code	2	32	4096	4096	1024	0.90	0.90	—	0.95	—
LLaDA2.1-Mini											
SingleBD (Native)	Math	1	32	4096	4096	1024	—	—	—	0.70	0.50
SingleBD (Native)	Code	1	32	4096	4096	1024	—	—	—	0.70	0.50
MultiBD (training-free)	Math	2	32	4096	4096	1024	0.10	0.90	—	0.70	0.50
MultiBD (training-free)	Code	2	32	4096	4096	1024	0.90	0.90	—	0.70	0.50

refinement, while future DUMMY slots remain invisible until activated. As shown in Figure 7(3), this separates cached prefix blocks from active *Block Buffer* slots, enabling prefix KV reuse while still refining multiple active blocks in parallel.

D Experimental Details

This appendix reports the inference and MultiTF post-training hyperparameters used in our experiments. “—” indicates that the corresponding hyperparameter is not applicable. SingleBD (Native) denotes the original single-block inference of each BD-LM; MultiBD (training-free) denotes MultiBD inference without post-training; MBD-* denotes the corresponding MultiTF-post-trained model.

Table 5 MultiTF post-training hyperparameters. t_{low} and t_{high} denote the mask-ratio range; ρ is the margin ratio used to determine the effective upper bound t_{eff} ; N_{rand} is the number of random *group-layouts* per sample. The random-scheduler ablation uses a separate power-law bias γ_{rand} , which is independent of ρ and is not used in the chain-uniform scheduler.

Target Model	Task	Objective	Data	Seq Len	Block	Max Group	t_{low}	t_{high}	ρ	N_{rand}	Steps
MBD-LLaDA2-Mini-DMax	Math	MultiTF + DMax OPUT	60k	2048	32	2	0.001	1.00	ρ_{cfg}	0	15000
MBD-LLaDA2-Mini-DMax	Code	MultiTF + DMax OPUT	60k	2048	32	2	0.001	1.00	ρ_{cfg}	2	4000
MBD-LLaDA2-Mini	Math	MultiTF CE	60k	2048	32	2	0.001	1.00	ρ_{cfg}	0	15000
MBD-LLaDA2-Mini	Code	MultiTF CE	60k	2048	32	2	0.001	1.00	ρ_{cfg}	0	6500
MBD-SDAR-8B-Chat-b32	Math	MultiTF CE	20k	2048	32	4	0.001	1.00	ρ_{cfg}	3	3125
MBD-SDAR-8B-Chat-b32	Code	MultiTF CE	10k	2048	32	4	0.001	1.00	ρ_{cfg}	3	1670
MBD-SDAR-8B-Chat-b4	Math	MultiTF CE	20k	2048	4	4	0.001	1.00	ρ_{cfg}	2	1250
MBD-SDAR-8B-Chat-b4	Code	MultiTF CE	10k	2048	4	4	0.001	1.00	ρ_{cfg}	2	200